



HARJOITUSTYÖN TYÖRAPORTTI

Scooter Fleet Management

TEKIJÄ/T Sauli Pitkäkangas

SISÄLTÖ

1	JOHDANTO.....	4
2	ENTITEETIT JA TIETOMALLI	5
2.1	Station	5
2.2	Scooter	5
2.3	ScooterDetail	6
2.4	Feature	6
2.5	Ride	7
2.6	AppUser.....	7
3	RELAATIOT	9
3.1	1:1 relaatiot.....	9
3.2	1:N relaatiot	9
3.3	M:N relaatiot	9
4	CRUD-TOIMINNOT	10
4.1	Yleinen toteutus.....	10
4.2	Listaaaminen	10
4.3	Lisääminen	11
4.4	Muokkaaminen	12
4.5	Poistaminen.....	13
4.6	Relaatiot CRUD-toiminnoissa	13
4.7	Tietokantaan tallentaminen	13
5	SUODATTAMINEN (CRITERIA API).....	16
5.1	Dynaamiset predikaatit.....	16
5.2	JOIN relaatio	16
5.3	OR-ehto (X OR Y) AND Z	17
5.4	Päivämäärähaku	17
5.5	Osittainen tekstihaku (LIKE).....	18
5.6	Lajittelu	19
5.7	Käyttöliittymä	19
6	TYYLIT JA ULKOASU	21
6.1	Globaalit tyylit	21
6.2	Komponenttien tyylien muokkaus	21
6.3	Näkymäkohtainen CSS	23

6.4	LumoUtility-luokat.....	24
6.5	Hover-, focus- ja transition	24
7	ULKOASU JA RAKENNE (SPA).....	26
7.1	MainLayout rakenne	26
7.2	Navigaatio	26
7.3	Footer	27
7.4	Kolme erityyppistä näkymää	28
7.4.1	Etusivu – dashboard-tyylinen näkymä	28
7.4.2	Ajot – listaus, suodatus ja muokkaus samalla näkymällä	29
7.4.3	Potkulaudat – informaatio + grid + dialog	30
7.4.4	Yhteenveto näkymien eroista	31
8	AUTENTIKOINTI JA TIETOTURVA.....	32
8.1	Spring Securityn käyttöönotto	32
8.2	Käyttäjä entiteetin luominen	34
8.3	Roolikohtaiset sivunäkymät.....	36
8.4	Rekisteröitymissivu.....	37
8.5	Kustomoitu virheviesti	39
8.6	Oman kuvan lisääminen.....	41
9	MUUT TOIMINNALLISUUDET	44
9.1	GitHub-julkaisu	44
9.2	Lokalisointi.....	44
9.3	Docker-image ja Dockerfile	45
9.4	Docker-Compose	47
9.5	Tiedoston lataus ja tallennus.....	49
10	POHDINTA.....	50
	LÄHTEET	52

1 JOHDANTO

GitHub-linkki: <https://github.com/Cyp6ty/vaadin-scooter-app>

Tämän harjoitustyön tavoitteena oli toteuttaa Vaadin- ja Spring Boot -teknologioihin perustuva web-sovellus, jossa yhdistyvät tietokannan käyttö, useat entiteetit, relaatiot, CRUD-toiminnot sekä käyttöliittymän suunnittelu. Työ toteutettiin kurssin vaatimusten pohjalta, joissa painotettiin sekä teknistä toimivuutta että käyttöliittymän rakenteen ja ulkoasun suunnittelua.

Harjoitustyön aiheeksi valitsin sähköpotkulautojen hallintajärjestelmän: Scooter Fleet Management. Sovelluksen tarkoituksena on mahdollistaa potkulautojen, asemien, ajojen ja muihin niihin liittyvien tietojen hallinnan yhden käyttöliittymän kautta työntekijän näkökulmasta katsottuna. Valitsin tämän aiheen, koska sähköpotkulautojen kanssa työskentely on minulle tuttua, ja se soveltui hyvin relaatio-tietokantaa hyödyntävään sovellukseen ja mahdollisti useiden erilaisten näkymien sekä käytännöllisten toiminnallisuuksien toteuttamisen.

Projektin aikana sovellukseen toteutettiin useita entiteettejä ja niiden välisiä suhteita, kuten 1:1-, 1:N- ja M:N-relaatioita. Lisäksi jokaiselle pääentiteetille rakennettiin listaus-, lisäys-, muokkaus- ja poistotoiminnot. Käyttöliittymä toteutettiin yksisivuisena SPA-ratkaisuna, jossa näkymät toimivat yhteisen MainLayout-rakenteen sisällä. Työssä panostettiin myös käyttöliittymän visuaalisuuteen, navigointiin, joissakin komponenteissa responsiivisuuteen sekä sovelluksen yleiseen käytettävyyteen.

Työhön sisällytettiin lisäksi useita jatko-ominaisuuksia, kuten Criteria API:lla toteutettu edistynyt haku, lokalisointi, Docker-käyttöönotto sekä GitHub-julkaisu. Kaikkia tehtävänannon lisäominaisuuksia en ehtinyt toteuttaa, mutta tavoitteena oli saada mahdollisimman laaja kokonaisuus (ainakin 30 vaatimusta) valmiiksi annetussa ajassa. Projekti toimi käytännön harjoituksena Full stack -sovelluskehityksestä Java-ympäristössä.

Tässä raportissa esitellään työn toteutus vaiheittain. Raportissa käydään läpi sovelluksen tietomalli, CRUD-ratkaisut, hakutoiminnot, käyttöliittymärakenne, toteutetut lisäominaisuudet sekä projektin aika tehdyt tekniset ratkaisut.

Sovelluksen käynnistykseen yhteydessä luodaan oletuskäyttäjät, joiden käyttäjätunnukset ja salasanat alapuolella:

1. User – user123
2. Super – super123
3. Admin – admin123

2 ENTITEETIT JA TIETOMALLI

2.1 Station

Station-entiteetti kuvaa sähköpotkulautojen pysäköintiasemaa järjestelmässä.

Validoitavat kentät ovat:

- name
- city
- address
- capacity
- area

Capacity määrittelee, kuinka monta potkulautaa asemalle voidaan enintään sijoittaa ja *area* määrittelee missä kaupungin osassa asema on.

Station liittyy Scooter-entiteettiin 1:N-relaatiolla, eli yhdellä asemalla voi olla useita potkulautoja.

```
// 1:M - Yhdellä stationilla voi olla monta scooteria
// KÄÄNTEINEN PUOLI
@OneToMany(mappedBy = "station") 2 usages
private List<Scooter> scooters = new ArrayList<>();
```

Kuva 1. Relaatio Station.java tiedostossa

2.2 Scooter

Scooter-entiteetti kuvaa yksittäistä sähköpotkulautaa järjestelmässä.

Validoitavat kentät ovat:

- model
- serialNumber
- batteryLevel
- status
- manufactureYear

Sarjanumero on määritelty uniikiksi, jotta jokainen lauta voidaan yksilöidä. Akun varaustaso on rajattu välille 0–100.

Scooter liittyy Station-entiteettiin ManyToOne -relaatiolla, useat laudat voivat kuulua samalle asemalle. Lisäksi scooterilla on 1:1-relaatio ScooterDetail-entiteettiin sekä M:N -relaatio Feature-entiteettiin.

```
// M:1 - Yhdellä scooterilla on yksi station
// Yhdellä stationilla voi olla monta scooteria
// OMISTAVA PUOLI
@ManyToOne 2 usages
@JoinColumn(name = "station_id")
private Station station;
```

Kuva 2. Relaatio Scooter- ja Station-entiteettien väliltä Scooter.java tiedostossa.

2.3 ScooterDetail

ScooterDetail-entiteetti sisältää potkulaudan tekniset lisätiedot.

Validoitavat kentät ovat:

- firmwareVersion
- maxSpeed
- lastInspectionDate
- qrCode
- weight

QR-koodi on määritelty uniikiksi, jotta jokainen potkulauta voidaan tunnistaa huoltoa tai käyttöä varten.

ScooterDetail liittyy Scooter-entiteettiin 1:1-relaatiolla.

```
// 1:1 - Yhdellä ScooterDetaililla on yksi scooter
// OMISTAVA PUOLI
@OneToOne 2 usages
@JoinColumn(name = "scooter_id", unique = true)
private Scooter scooter;
```

Kuva 3. Relaatio ScooterDetailin ja Scooterin väliltä ScooterDetail.java tiedostossa.

2.4 Feature

Feature-entiteetti kuvaa sähköpotkulaudan lisättäviä ominaisuuksia.

Validoitavat kentät ovat:

- name
- description
- category
- active
- installationCost

Feature liittyy Scooter-entiteettiin M:N -relaatiolla, eli yhdellä potkulaudalla voi olla useita ominaisuuksia ja sama ominaisuus voi kuulua usealle potkulaudalle.

```
// M:N - Yhdellä scooterilla voi olla monta ominaisuutta
// Yksi ominaisuus voi kuulua monelle scooterille
// KÄÄNTEINEN PUOLI
@ManyToMany(mappedBy = "features") 3 usages
private Set<Scooter> scooters = new HashSet<>();
```

Kuva 4. Relaatio Featuren ja Scooterin väliltä Feature.java tiedostossa.

2.5 Ride

Ride-entiteetti lisättiin täydentämään sovelluksen toiminnallisuutta, vaikka tehtävänannon minimivaatimus olisi täyttynyt jo neljällä entiteetillä. Ride-entiteetti kuvaa yksittäistä ajokertaa.

Validoitavat kentät ovat:

- startTime
- endTime
- distanceKm
- price
- status

Ajan kentät ovat rajattu siten, että ajon aika ei voi olla tulevaisuudessa. Lisäksi matkan ja hinnan tulee olla positiivisia arvoja.

Ride liittyy useisiin muihin entiteetteihin. Jokaisella ajolla on yksi scooter sekä lähtö- ja pääteasema. Tämä on toteutettu ManyToOne -relaatiolla Scooter- ja Station-entiteetteihin.

```
// N:1 - Monta ajoa voi kuulua yhdelle scooterille
// OMISTAVA PUOLI
// Ride -> Scooter
@NotNull 3 usages
@ManyToOne
@JoinColumn(name = "scooter_id")
private Scooter scooter;
```

Kuva 5. Relaatio Riden ja Scooterin välillä Ride.java tiedostossa.

2.6 AppUser

AppUser on kuudes entiteetti ja se kuvaa sovelluksen käyttäjää ja sitä käytetään autentikointiin sekä käyttäjäkohtaisten tietojen tallentamiseen.

Entiteetti sisältää kentät:

- username
- passwordHash
- roles
- profileImagePath

Salasanaa ei tallenneta selkokielisenä, vaan se muunnetaan hash-muotoon ennen tallennusta.

AppUser-entiteettiä käytetään yhdessä Spring Securityn kanssa käyttäjän tunnistamiseen ja käyttö-oikeuksien hallintaan.

Entiteetti liittyy erityisesti raportin kohtaan: 8 AUTENTIKOINTI JA TIETOTURVA - [Käyttäjä entiteetin luominen](#).

3 RELAATIOT

Relaatiot on toteutettu JPA:n avulla käyttäen annotaatioita: *@OneToOne*, *@OneToMany*, *@ManyToOne* ja *@ManyToMany*.

Sovelluksessa on käytössä kolme eri relaatiotyyppiä:

- 1:1 relaatiot
- 1:N relaatiot
- M:N relaatiot

Tietomalli ja entiteettien relaatiot:

4. Station 1 --- N Scooter
5. Scooter 1 --- 1 ScooterDetail
6. Scooter N --- N Feature
7. Scooter 1 --- N Ride
8. Station 1 --- N Ride

3.1 1:1 relaatiot

1:1-relaatio on toteutettu Scooter- ja ScooterDetail -entiteettien välillä.

Jokaisella potkulaudalla voi olla yksi tekninen lisätietorivi, joka sisältää firmware-version, QR-koodin, sekä viimeisen tarkastuspäivän. Tämä on toteutettu siten, että ScooterDetail sisältää vierasavaimen *scooter_id*, joka viittaa Scooter-entiteettiin.

Tämä mahdollistaa teknisten tietojen erottamisen pääentiteetiltä omaan tauluunsa, selkeyttäen tietomallia.

3.2 1:N relaatiot

Sovelluksessa käytetään useita 1:N-relaatioita entiteettien välillä.

Station- ja Scooterin välillä on 1:N-relaatio, jossa yhdellä asemalla voi olla useita potkulautoja. Jokainen lauta kuuluu kuitenkin yhdelle asemalle.

Lisäksi Ride-entiteetti liittyy Scooter- ja Station -entiteetteihin 1:N relaatiolla. Yhdellä laudalla voi olla useita ajokertoja, ja yhdeltä asemalta voi alkaa tai päättyä useita ajoja.

3.3 M:N relaatiot

Scooter- ja Feature-entiteettien välillä on M:N-relaatio.

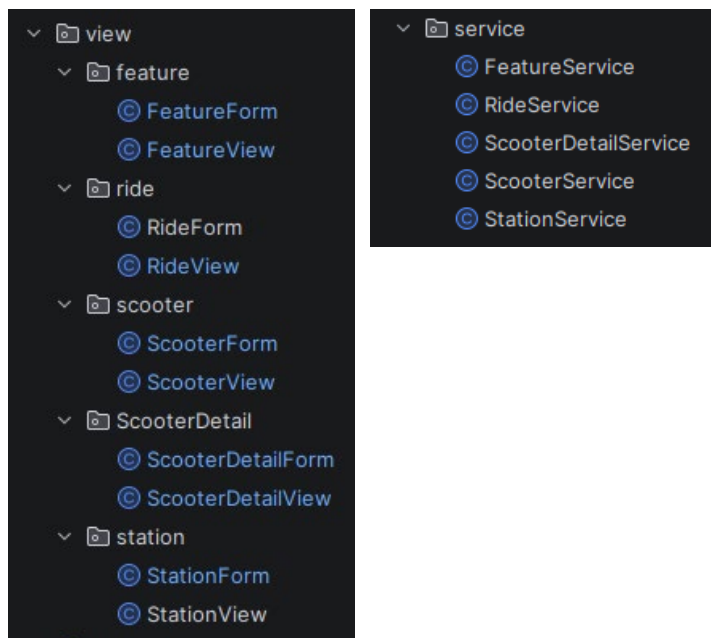
Yhdellä potkulaudalla voi olla useita ominaisuuksia, ja sama ominaisuus voi kuulua usealle potkulaudalle. Relaatio on toteutettu liitostaulun avulla, joka sisältää viittaukset sekä scooter- että feature-entiteetteihin.

Tämä rakenne mahdollistaa ominaisuuksien joustavan liittämisen eri potkulautoihin ilman tietojen dublikoimista.

4 CRUD-TOIMINNOT

Sovelluksen CRUD-toiminnot on toteutettu kaikille entiteeteille (Station, Scooter, ScooterDetail, Feature ja Ride) samalla tavalla. Kaikki noudattavat samaa rakennetta.

CRUD-toiminnot on tehty käyttäliittymältä tietokantaan asti käyttäen näkymiä View, lomakkeita Form, palveluluokkia Service, sekä JPA-repositoryitä.



Kuvat 6 ja 7. Kansio- ja tiedostorakenteet Service, Form ja View -tiedosta.

4.1 Yleinen toteutus

Jokaiselle entiteetille on luotu:

- listaussivu (Grid)
- lomake uuden tiedon lisäämiseen
- sama lomake käyttää olemassa olevan tiedon muokkaamiseen
- poistotoiminto
- repository tietokanta käsittelyyn

Käyttäliittymä (View-luokat) kommunikoi Service-luokkien kanssa, jotka käyttävät luokkakohtaista repositorya tietokantaoperaatioiden suorittamiseen, erottaen käyttäliittymän ja tietokantakerroksen toisistaan.

4.2 Listaaminen

Kaikille entiteeteille on tehty listaussivu Grid-komponenteilla. Grid näyttää tietokannassa olevat rivit taulukkomuodossa.

Esimerkiksi Station näkymässä näytetään:

- id
- name
- city
- address

- capacity
- area

Muiden entiteettien kohdalla Grid näyttää vastaavasti niiden omat kentät sekä tarvittaessa relaatioiden tiedot. Esimerkiksi Ride-näkymässä listassa näkyvät relaatiot:

- potkulauta (scooter)
- lähtöasema (startStation)
- pääteasema (endStation)

Relaatiot näkyvät myös käyttöliittymässä, eikä pelkästään tietokannassa.

ID	Aloitusaika	Lopetusaika	Kilometri määrä	Ajon hinta	Status	Potkulauta	Aloitusasema	Pääteasema	Aloitusaika •
1	2026-03-10T...	2026-03-10T...	3.5	5.9	COMPLETED	SN1001 - Ni...	Tori	Teatteri	
2	2026-03-10T...	2026-03-10T...	2.8	4.8	COMPLETED	SC-2002 - Xi...	Teatteri	Savonia	
3	2026-03-11T...	2026-03-11T...	4.2	6.7	COMPLETED	SC-2003 - Se...	Savonia	Tori	
4	2026-03-12T...	2026-03-12T...	10.1	10.2	COMPLETED	SC-3001 - A...	Tori	Teatteri	
5	2026-03-16T...	2026-03-16T...	4.0	3.4	COMPLETED	SC-1002 - Ni...	Teatteri	Savonia	

Lopetusaika •

Kilomerimäärä •

Status •
Valitse ajon status

Hinta •

Potkulauta •
Valitse potkulauta

Lähtöasema •
Valitse lähtöasema

Pääteasema •
Valitse pääteasema

Tallenna
Poista
Peruuta

Kuva 8. Entiteettien listaus (Ride-näkymä), jossa näkyvät myös relaatiot (Scooter, aloitusasema ja pääteasema).

4.3 Lisääminen

Uuden tiedon lisääminen tapahtuu lomakkeella (Form), joka avataan käyttöliittymässä painikkeella "Lisää uusi...".

Sovelluksen lomakkeissa on käytetty kenttiä kuten:

- TextField (String)
- TextArea (String)
- IntegerField (Integer)
- NumberField (Double)
- ComboBox (String, Boolean, Entity)
- MultiSelectComboBox (Entity)
- DatePicker
- DateTimePicker

ja Binder-luokkaa, joka sitoo lomakekentät suoraan entiteetin kenttiin.

Malli •

Syötä malli

Sarjanumero •

0

Sarjanumeron tulee olla vähintään 4 merkkiä

Akun varaustaso •

-2

Varaustaso tulee olla väliä 0-100

Status •

Valitse status

Valitse status

Valmistusvuosi •

Syötä valmistusvuosi

Asema •

Valitse asema

Valitse asema

Ominaisuudet

Tallenna

Poista

Peruuta

Kuva 9. Esimerkki yhdestä sovelluksen lomakkeesta (ScooterForm), jossa validointi estää virheelliset arvot.

Jokaiselle lomakkeen kentälle on annettu kenttäkohtaiset säännöt jokaisen listaussivun lomakkeissa. Esimerkiksi yksikään kenttä ei voi olla tyhjä, aikakentät eivät hyväksy tulevaisuuden arvoja eikä menneisyys voi olla tulevaisuudessa, tekstikentät ovat rajattu merkkien vähimmäispituudella + muita tarkistuksia. Jos käyttäjä antaa vääränlaisen syötteen, lomake kertoo siitä saman tien.

Kun käyttäjä täyttää lomakkeen ja painaa tallennuspainiketta:

1. Binder validoi syötetyt tiedot
2. tiedot kirjoitetaan entiteettioliioon
3. Service-luokka tallentaa olion repositoryn kautta tietokantaan.

4.4 Muokkaaminen

Muokkaaminen tapahtuu klikkaamalla grid taulukossa olevaa riviä.

Valittu entiteetti avataan lomakkeelle, jossa sen tiedot näytetään käyttäjälle. Käyttäjä voi muokata tiedot annettujen ohjeiden (validointien asetusten) mukaisesti.

Tallennus tapahtuu samalla tavalla kuin lisäyksessä:

- Binder validoi ja päivittää entiteettioliion
- Service tallentaa muutokset tietokantaan

4.5 Poistaminen

Poistaminen tapahtuu valitsemalla rivi grid-taulukosta ja painamalla lomakkeen "Poista"-painiketta.

Tämän jälkeen:

- Service-luokka kutsuu kyseisen entiteetin repositoryn delete-metodia
- tietue poistetaan tietokannasta
- Grid päivitetään näyttämään ajantasainen tilanne

4.6 Relaatiot CRUD-toiminnoissa

CRUD-toiminnot tukevat myös entiteettien välisiä relaatioita.

Relaatiot näkyvät käyttöliittymässä seuraavasti:

- **1:1** (Scooter $\leftarrow \rightarrow$ ScooterDetail): ScooterDetail-listassa näkyy siihen liittyvä potkulauta
- **1:N** (Station $\leftarrow \rightarrow$ Scooter): Scooter-näkymässä näkyy, mille asemalle potkulauta kuuluu
- **M:N** (Scooter $\leftarrow \rightarrow$ Feature): Scooter-lomakkeella voidaan valita useita ominaisuuksia yhdelle potkulaudalle
- Ride-entiteetissä näkyvät sekä potkulauta että lähtö- ja pääteasema listauksessa

Ride-näkymä toimii hyvänä esimerkkinä relaatioiden esittämisestä UI:ssa, koska siinä yhdistyy useita entiteettejä samaan näkymään.

4.7 Tietokantaan tallentaminen

Kaikki CRUD-operaatiot toimivat käyttöliittymältä tietokantaan asti:

- uusi tieto ilmestyy grid-taulukkoon tallennuksen jälkeen
- muokatut tiedot päivittyvät käyttöliittymässä
- poistettu tieto häviää listasta
- tiedot ovat nähtävissä myös H2-tietokannassa

Run

Run Selected

Auto complete

Clear

SQL statement:

SELECT * FROM RIDE

SELECT * FROM RIDE;

ID	DISTANCE_KM	END_TIME	PRICE	START_TIME	STATUS	END_STATION_ID	SCOOTER_ID	START_STATION_ID
1	3.5	2026-03-10 08:25:00	5.9	2026-03-10 08:10:00	COMPLETED	2	1	1
2	2.8	2026-03-10 12:18:00	4.8	2026-03-10 12:00:00	COMPLETED	3	2	2
3	4.2	2026-03-11 17:55:00	6.7	2026-03-11 17:40:00	COMPLETED	1	3	3
4	10.1	2026-03-12 10:17:00	10.2	2026-03-12 10:01:00	COMPLETED	2	4	1
5	4.0	2026-03-16 12:10:00	3.4	2026-03-16 12:00:00	COMPLETED	3	5	2

(5 rows, 0 ms)

Kuva 10. Tallennettu data H2-tietokannassa (Ride-taulu).

```

18 public class ScooterDetailView extends VerticalLayout {
47     // Määrittää listan
48     private void configureGrid() { 1 usage  SaleSan *
49         grid.addColumn(ScooterDetail::getId).setHeader("ID");
50         grid.addColumn(ScooterDetail::getFirmwareVersion).setHeader("Ohjelmistoversio");
51         grid.addColumn(ScooterDetail::getMaxSpeed).setHeader("Maksimi nopeus");
52         grid.addColumn(ScooterDetail::getLastInspectionDate).setHeader("Viim. tarkastuspäivä");
53         grid.addColumn(ScooterDetail::getQrCode).setHeader("QR-koodi");
54         grid.addColumn(ScooterDetail::getWeight).setHeader("Paino (kg)");
55
56         grid.addColumn(ScooterDetail detail ->
57             detail.getScooter() != null
58             ? detail.getScooter().getSerialNumber() + " - " + detail.getScooter().getModel()
59             : "")
60             .setHeader("Potkulauta");
61
62         grid.setSizeFull();
63         grid.asSingleSelect().addValueChangeListener(
64             ComponentValueChangeEvent<Grid<...>, ScooterDetail> event -> editDetail(event.getValue()));
65     }
66
67     // Määrittää buttonit
68     private void configureForm() { 1 usage  SaleSan
69         form.setWidth("25em");
70
71         form.getSaveButton().addClickListener( ClickEvent<Button> event -> saveDetail());
72         form.getDeleteButton().addClickListener( ClickEvent<Button> event -> deleteDetail());
73         form.getCancelButton().addClickListener( ClickEvent<Button> event -> closeEditor());
74     }
75

```

Kuva 11. CRUD-toimintojen toteutus ScooterDetailView -luokassa.

```

// Hakee tiedot tietokannasta
private void updateList() { grid.setItems(scooterDetailService.findAll()); }

// Lisää uuden
private void addDetail() { 1 usage  SaleSan
    grid.asSingleSelect().clear();
    form.setScooterDetail(new ScooterDetail());
    form.setVisible(true);
}

// Avaa valitun muokattavaksi
private void editDetail(ScooterDetail scooterDetail) { 1 usage  SaleSan
    if (scooterDetail == null) {
        closeEditor();
    } else {
        form.setScooterDetail(scooterDetail);
        form.setVisible(true);
    }
}

// Tallentaa muutokset
private void saveDetail() { 1 usage  SaleSan
    ScooterDetail detail = form.getScooterDetail();

    if (detail != null && form.writeScooterDetail()) {
        scooterDetailService.save(detail);
        updateList();
        closeEditor();
        Notification.show(text: "Uusi lisätieto tallennettu");
    }
}

```

Kuva 12. Haku, lisäys, muokkaaminen ja tallentaminen ScooterDetailView -luokassa.

```
// Poistaa valitun
private void deleteDetail() { 1 usage  👤 SaleSan
    ScooterDetail detail = form.getScooterDetail();

    if (detail != null && detail.getId() != null) {
        scooterDetailService.delete(detail);
        updateList();
        closeEditor();
        Notification.show(text: "Lisätieto poistettu");
    }
}
```

Kuva 13. Poistaminen ScooterDetailView -luokassa.

5 SUODATTAMINEN (CRITERIA API)

Sovellukseen toteutettiin edistynyt hakutoiminnallisuus JPA:n Criteria API:a käyttäen. Hakutoiminto rakennettiin niin, että käyttäjä voi syöttää useita eri hakuehtoja käyttöliittymässä, ja niiden perusteella muodostetaan dynaaminen tietokantakysely.

Hakutoiminnallisuus on toteutettu erillisellä RideFilter-luokalla, joka sisältää kaikki mahdolliset hakuehdot, sekä RideRepositoryCustomImpl-luokalla, jossa Criteria API-kysely rakennetaan näiden ehtojen perusteella.

Käyttöliittymässä käyttäjä voi hakea ajoja esimerkiksi hakusanan, statuksen, lähtöaseman, päivämäärävalin sekä lajittelun perusteella. Nämä arvot välitetään RideView-luokasta RideService-luokan kautta repositoryyn, jossa itse haku suoritetaan.

5.1 Dynaamiset predikaatit

Hakutoiminto tehtiin niin, että predikaatteja lisätään kyselyyn vain silloin, kun käyttäjä on syöttänyt kyseisen hakuehdon. Tämä toteutetaan tarkistamalla jokainen kenttä erikseen ennen kuin se lisätään predicates-listaan.

Esimerkiksi statuksen suodatus lisätään vain, jos käyttäjä on valinnut statuksen:

```
if (filter.getStatus() != null && !filter.getStatus().isEmpty()) {
    predicates.add(cBuilder.equal(ride.get("status"), filter.getStatus()));
}
```

Kuva 14. Dynaamisten predikaattien muodostaminen RideRepositoryCustomImpl-luokassa.

Vastaava logiikka on käytössä myös muiden hakuehtojen, kuten päivämäärän, aseman ja hakusanan kohdalla. Käyttäjä voi tehdä hakuja yhdellä tai useammalla ehdolla ilman, että kaikkia kenttiä tarvitsee täyttää.

5.2 JOIN relaatio

Haussa käytetään JOIN-operaatiota, jotta voidaan hakea tiedot relaatioihin liittyvistä entiteeteistä.

Esimerkiksi lähtöaseman perusteella suodattaminen tehdään JOINin avulla:

```
Join<Ride, Station> startStationJoin = ride.join(s: "startStation");
```

```
if (filter.getStartStation() != null) {
    predicates.add(cBuilder.equal(
        startStationJoin.get("id"),
        filter.getStartStation().getId()
    ));
}
```

Kuva 15. JOIN hakuehdoissa (Station ja Scooter) RideRepositoryCustomImpl-luokassa.

Tämän avulla voidaan hakea kaikki ajot, jotka liittyvät tiettyyn asemaan, vaikka asema sijaitsee eri taulussa kuin Ride.

Lisäksi hakusanahaussa käytetään JOINia myös potkulaudan ja pääteaseman tietoihin.

5.3 OR-ehto (X OR Y) AND Z

Hakutoiminnossa toteutettiin monimutkainen OR-ehto, jossa hakusana kohdistuu useaan eri kenttään.

Hakusana haetaan kentistä:

- potkulaudan malli
- potkulaudan sarjanumero
- lähtöaseman nimi
- pääteaseman nimi

```
if (filter.getKeyword() != null && !filter.getKeyword().isEmpty()) {
    String pattern = "%" + filter.getKeyword().toLowerCase() + "%";

    Predicate model = cBuilder.like(
        cBuilder.lower(scooterJoin.get("model")),
        pattern
    );

    Predicate serialNumber = cBuilder.like(
        cBuilder.lower(scooterJoin.get("serialNumber")),
        pattern
    );

    Predicate startStationName = cBuilder.like(
        cBuilder.lower(startStationJoin.get("name")),
        pattern
    );

    Join<Ride, Station> endStationJoin = ride.join(s: "endStation");
    Predicate endStationName = cBuilder.like(
        cBuilder.lower(endStationJoin.get("name")),
        pattern
    );

    // (model LIKE ? OR serialNumber LIKE ? OR station LIKE ?)
    Predicate orCondition = cBuilder.or(
        model,
        serialNumber,
        startStationName,
        endStationName
    );
}
```

Kuva 16. OR-ehdon toteutus hakusanahaussa RideRepositoryCustomImpl-luokassa.

Tämä yhdistetään muihin ehtoihin AND-logiikalla, jolloin lopullinen kysely on muotoa:
(model LIKE ? OR serialNumber LIKE ? OR station LIKE ?)

AND = status ?

AND = startTime BETWEEN ?

5.4 Päivämäärähaaku

Hakutoiminto tukee myös päivämäärävälin perusteella suodattamista. Käyttäjä voi syöttää esimerkiksi:

- pelkän alkupäivän
- pelkän loppupäivän
- tai molemmat

Alkupäivä muunnetaan päivän alkuun ja loppupäivä päivän loppuun:

```
// alkupäivä
if (filter.getFromDate() != null) {
    LocalDateTime fromDateTime = filter.getFromDate().atStartOfDay();
    predicates.add(cBuilder.greaterThanOrEqualTo(
        ride.get("startTime"),
        fromDateTime
    ));
}

// loppupäivä
if (filter.getToDate() != null) {
    LocalDateTime toDateTime = filter.getToDate().atTime(hour: 23, minute: 59, second: 59);
    predicates.add(cBuilder.lessThanOrEqualTo(
        ride.get("startTime"),
        toDateTime
    ));
}
```

Kuva 17. Päivämääräehtojen toteutus RideRepositoryCustomImpl-luokassa.

Näiden perusteella muodostetaan ehdot:

- cBuilder.greaterThanOrEqualTo(...)
- cBuilder.lessThanOrEqualTo(...)

mahdollistaen tarkat haut tietyllä aikavälillä.

5.5 Osittainen tekstihaku (LIKE)

Hakusana toteutettiin käyttäen LIKE-operaatiota, joka mahdollisti osittaisen tekstihaun.

Hakusana muunnetaan pieniksi kirjaimiksi ja siihen lisätään jokerimerkit '%':

```
if (filter.getKeyword() != null && !filter.getKeyword().isEmpty()) {
    String pattern = "%" + filter.getKeyword().toLowerCase() + "%";

    Predicate model = cBuilder.like(
        cBuilder.lower(scooterJoin.get("model")),
        pattern
    );
}
```

Kuva 18. LIKE-haun toteutus RideRepositoryCustomImpl-luokassa.

Käyttäjä voi hakea esimerkiksi osalla sanasta, kuten "nine" → "Ninebot", tai vastaavasti jotain asemaa esimerkiksi "teri" → "Teatteri". Jos sama merkkijono ilmenee jossain muussa kohdassa, esim. scooterin nimessä, haku palauttaa myös ne.

5.6 Lajittelu

Tulokset voidaan järjestää käyttäjän valinnan mukaan. Lajittelu toteutettiin sortBy-kentän perusteella.

```
if (filter.getSortBy() != null) {
    switch (filter.getSortBy()) {
        case "startTimeAsc":
            cQuery.orderBy(cBuilder.asc(ride.get("startTime")));
            break;
        case "startTimeDesc":
            cQuery.orderBy(cBuilder.desc(ride.get("startTime")));
            break;
        case "priceAsc":
            cQuery.orderBy(cBuilder.asc(ride.get("price")));
            break;
        case "priceDesc":
            cQuery.orderBy(cBuilder.desc(ride.get("price")));
            break;
        default:
            cQuery.orderBy(cBuilder.desc(ride.get("startTime")));
    }
} else {
    // oletus
    cQuery.orderBy(cBuilder.desc(ride.get("startTime")));
}

return entityManager.createQuery(cQuery).getResultList();
```

Kuva 19. Lajittelun toteutus RideRepositoryCustomImpl-luokassa.

Jos käyttäjä ei valitse lajittelua, näytetään oletuksena uusimmat ajot ensin. Lajittelussa voidaan käyttää ajonaloitusaikaa ja hintaa nousevassa tai laskevassa järjestyksessä.

5.7 Käyttöliittymä

Hakutoiminnallisuus toteutettiin RideView-luokassa. Käyttäjä syöttää haku ehdot käyttöliittymässä, jonka jälkeen ne kerätään RideFilter-olioon:

```
private RideFilter buildFilter() { 1 usage new *
    RideFilter filter = new RideFilter();

    filter.setKeyword(keyword.getValue());
    filter.setStatus(status.getValue());
    filter.setStartStation(startStation.getValue());
    filter.setFromDate(fromDate.getValue());
    filter.setToDate(toDate.getValue());
    filter.setSortBy(sortBy.getValue());
    return filter;
}
```

Kuva 20. Hakutoiminnon käyttö RideView-luokassa.

Hakupainike kutsuu service-luokkaa:

```
searchButton.addClickListener( ClickEvent<Button> event ->
    grid.setItems(rideService.searchRides(buildFilter())));
```

Kuva 21. Hakupainikkeen kutsu.

Ja Tyhjennä-painike palauttaa alkuperäisen listan.

```
private void clearFilters() { 1 usage new *
    keyword.clear();
    status.clear();
    startStation.clear();
    fromDate.clear();
    toDate.clear();
    sortBy.clear();

    grid.setItems(rideService.findAll());
}
```

Kuva 22. Tyhjennä-painikkeen toiminta.

Hakusana	Status	Lähtöasema	Ajon aloituspäivä	Ajon lopetuspäivä	Lajittelu			
Hae esim. scooter, asen...	Valitse status	Valitse lähtöasema			Valitse lajittelu	Hae	Tyhjennä	
ID	Aloitusaika	Lopetusaika	Kilometri määrä	Ajon hinta	Status	Potkulauta	Aloitusasema	Pääteasema
1	2026-03-10T08:10	2026-03-10T08:25	3.5	5.9	COMPLETED	SN1001 - Ninebot ...	Tori	Teatteri
2	2026-03-10T12:00	2026-03-10T12:18	2.8	4.8	COMPLETED	SC-2002 - Xiaomi P...	Teatteri	Savonia
3	2026-03-11T17:40	2026-03-11T17:55	4.2	6.7	COMPLETED	SC-2003 - Segway ...	Savonia	Tori
4	2026-03-12T10:01	2026-03-12T10:17	10.1	10.2	COMPLETED	SC-3001 - Augment	Tori	Teatteri
5	2026-03-16T12:00	2026-03-16T12:10	4.0	3.4	COMPLETED	SC-1002 - Ninebot	Teatteri	Savonia
6	2026-03-23T13:49	2026-03-23T14:05	10.0	5.0	CANCELLED	SN-9999 - Birdi	Teatteri	Savonia
7	2026-03-22T15:00	2026-03-22T15:10	2.0	0.6	FAILED	SC-2002 - Xiaomi P...	Kotivarasto	Teatteri
8	2026-03-18T08:05	2026-03-18T08:19	3.1	4.2	COMPLETED	SN1001 - Ninebot ...	Tori	Teatteri
9	2026-03-24T08:10	2026-03-24T08:24	3.2	4.3	COMPLETED	SN1001 - Ninebot ...	Tori	Teatteri
10	2026-03-12T15:46	2026-03-12T16:10	4.8	6.1	COMPLETED	SC-2002 - Xiaomi P...	S-market Keilankan...	Matkakeskus
11	2026-03-03T11:09	2026-03-03T12:01	3.9	8.6	COMPLETED	SC-3456 - Tier	Matkakeskus	Männistön tori
12	2026-03-07T17:06	2026-03-07T17:31	5.4	6.9	COMPLETED	SC-3001 - Augment	Tori	S-market Keilankan...

Kuva 23. Oletusnäkymä käyttöliittymässä RideView-luokasta ilman hakuehtoja.

Hakusana	Status	Lähtöasema	Ajon aloituspäivä	Ajon lopetuspäivä	Lajittelu			
ment	Valitse status	Valitse lähtöasema			Valitse lajittelu	Hae	Tyhjennä	
ID	Aloitusaika	Lopetusaika	Kilometri määrä	Ajon hinta	Status	Potkulauta	Aloitusasema	Pääteasema
4	2026-03-12T10:01	2026-03-12T10:17	10.1	10.2	COMPLETED	SC-3001 - Augment	Tori	Teatteri
12	2026-03-07T17:06	2026-03-07T17:31	5.4	6.9	COMPLETED	SC-3001 - Augment	Tori	S-market Keilankan...

Kuva 24. Hakusana "ment" tuotti 2 tulosta. (Malli: Augment)

6 TYYLIT JA ULKOASU

Käyttöliittymän ulkoasua muokattiin usealla eri tavalla käyttäen Vaadinin tarjoamia tyylityökaluja sekä omia CSS-määrittelyjä.

6.1 Globaalit tyylit

Sovellukseen määriteltiin globaalit tyylit styles.css-tiedostossa, jotka vaikuttavat koko sovellukseen.

Näiden tyylien avulla on:

- Vaihdettu sovelluksen fontti:
 - --lumo-font-family: "Segoe UI", Arial, sans-serif;
- Luotu oma väripaletti:
 - pääväri (--lumo-primary-color)
 - korostevärit (--app-secondary-color, --app-accent-color)
- Muokattu taustaa:
 - gradient-tausta koko sovellukselle (body-elementissä)
- Muokattu komponenttien ulkoasua:
 - nappien pyöristetyt kulmat (border-radius)
 - varjot (box-shadow)
 - glassmorphism-tyylinen tausta (backdrop-filter)
- Muokattu tekstikenttien ulkoasua:
 - läpinäkyvä tausta
 - blur-efekti
 - korostettu focus-tila

6.2 Komponenttien tyylien muokkaus

a) addClassName()

Tätä käytettiin RideView ja HomeView-luokissa.

Esimerkiksi RideView:issä komponentille lisättiin 2 omaa CSS-luokkaa, jotta tyylit saatiin kohdistettua tälle komponentille.

Ride-näkymässä käytettiin näitä:

```
filters.addClassName("ride-filter-bar");  
keyword.addClassName("ride-keyword-field");
```

Kuva 25. addClassName-luokan käyttö RideView.java tiedostossa

ja styles.css tiedostossa vastaavasti:

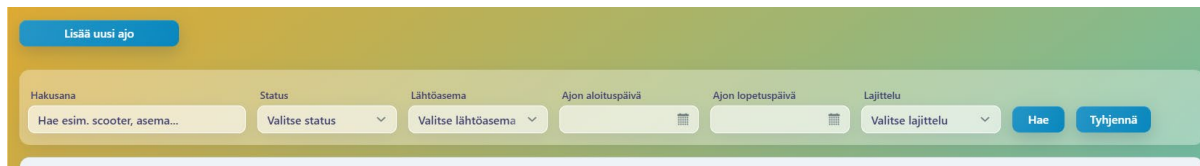
```

.ride-filter-bar {
  padding: 12px;
  border-radius: var(--lumo-border-radius-m);
  background: rgba(255, 255, 255, 0.34);
  box-shadow: 0 8px 24px rgba(31, 38, 135, 0.08);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
}

.ride-keyword-field {
  min-width: 300px;
}

```

Kuva 26. CSS-tyylit, jotka viittaavat RideView-luokan addClassName-luokkiin.



Kuva 27. RideView.java tiedostossa oleva filter bar.

b) `getStyle().set()`

Inline-tyylejä käytettiin myös RideView-luokassa `addNewButton`:in tyyliin:

```

addNewButton.getStyle().set("align-self", "flex-start");
addNewButton.getStyle().set("margin-bottom", "1rem");
addNewButton.getStyle().set("width", "220px");

```

Kuva 28. Tyylien käyttö suoraan java-koodissa RideView.java tiedostossa.

c) `addThemeVariants`

Vaadinin valmiita teema vaihtoehtoja käytettiin RideForm:in napeissa (tallenna, poista ja peruuta):

```

save.addThemeVariants(ButtonVariant.LUMO_PRIMARY, ButtonVariant.LUMO_SUCCESS);
delete.addThemeVariants(ButtonVariant.LUMO_ERROR);
cancel.addThemeVariants(ButtonVariant.LUMO_TERTIARY);

```

Kuva 29 Valmiiden teemojen käyttö RideForm.java tiedostossa.



Kuva 30. Nappien ulkoasu käyttöliittymässä "Ajot" -sivulla.

Näitä nappeja muokattiin vielä `styles.css` tiedostossa:

```
vaadin-button[theme~="success"] {
  background: var(--lumo-success-color);
}
vaadin-button[theme~="error"] {
  background: var(--lumo-error-color);
}
vaadin-button[theme~="tertiary"] {
  border: 1px solid rgba(255, 255, 255, 0.78);
  background: var(--lumo-tertiary-color);
}
```

Kuva 31. Button-komponenttien teemojen tarkempi muokkaus styles.css-tiedostossa, jotta sovelluksen globaali teema voitiin yliajaa.

6.3 Näkymäkohtainen CSS

Näkymäkohtainen CSS tehtiin etusivulle (HomeView.java)

Näkymälle lisättiin oma CSS-luokka: `addClassName("home-view");`

Kyseinen tyyli vaikuttaa useampaan komponenttiin samassa näkymässä:

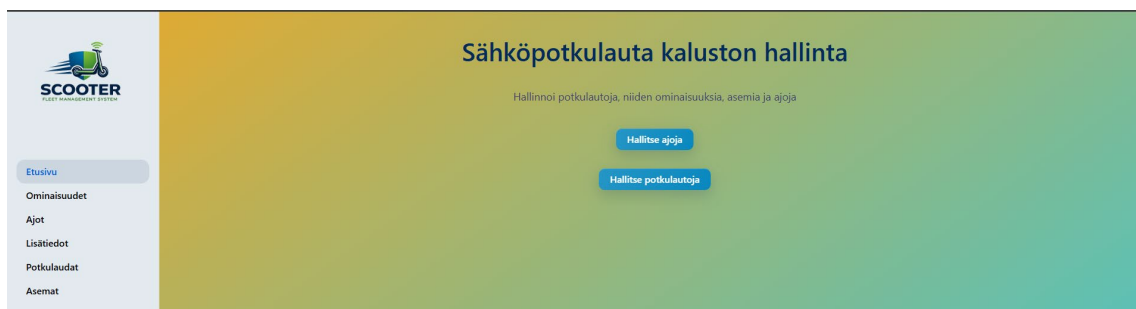
- otsikko h1
- teksti (p)
- painikkeet (vaadin-buttons)

Nämä sisältävät mm.:

- keskitetyn layoutin
- suuremman otsikkofontin
- väriteemaan mukaiset tekstivärit
- komponenttien välistyksen

```
/* HOMEVIEW */
.home-view {
  align-items: center;
  justify-content: center;
  text-align: center;
  padding: 40px;
}
.home-view h1 {
  font-size: 42px;
  color: var(--lumo-primary-color);
  margin-bottom: 10px;
}
.home-view p {
  font-size: 18px;
  color: var(--lumo-secondary-text-color);
}
.home-view vaadin-button {
  margin-top: 10px;
}
```

Kuva 32. CSS-määrittymiset styles.css luokassa HomeView.java tiedostoon.



Kuva 33. Etusivun ulkoasu HomeView.java -tiedostossa.

6.4 LumoUtility-luokat

Scooter-näkymässä on käytetty LumoUtility-luokkia infoBox-komponentin ulkoasun muokkaamiseen.

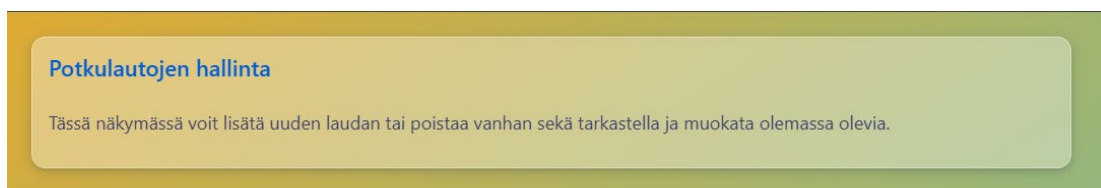
Tähän näkymään tehtiin "infoBox" -komponentti, jossa hyödynnettiin seuraavia utility-luokkia: TextColor, Padding, BorderRadius, Width, Margin ja FontSize.

```
infoBox.addClassName("glass-border");
infoBox.addClassNames(
    LumoUtility.MaxWidth.SCREEN_LARGE,
    LumoUtility.BackdropBlur.MEDIUM,
    LumoUtility.Padding.MEDIUM,
    LumoUtility.BoxShadow.SMALL,
    LumoUtility.Border.ALL,
    LumoUtility.BorderColor.CONTRAST_20,
    LumoUtility.BorderRadius.MEDIUM,
    LumoUtility.Margin.SMALL
);

title.addClassNames(
    LumoUtility.TextColor.PRIMARY,
    LumoUtility.FontSize.XLARGE
);

text.addClassNames(
    LumoUtility.TextColor.SECONDARY,
    LumoUtility.FontSize.LARGE
);
```

Kuva 34. Utility-luokat ScooterView.java -tiedostossa.



Kuva 35. InfoBox ScooterView.java -tiedostossa.

6.5 Hover-, focus- ja transition

Sovelluksen painikekomponenteille tehtiin CSS-määrittelyt eri tiloihin styles.css -tiedostossa. Painikkeiden yleisessä luokassa määriteltiin myös toiminto "transition: all 0.25s ease", joka löytyy vaa-din-button { ... transition: all 0.25s ease } -kohdasta.


```

/* Hover */
vaadin-button:hover {
  background: rgba(12, 140, 195, 0.77);
  border: 1px solid rgba(218, 218, 218, 0.69);
  box-shadow:
    0 12px 30px rgba(4, 43, 92, 0.3),
    inset 0 1px 1px rgba(255, 255, 255, 0.2);
  transform: translateY(-1px);
}

/* Focus */
vaadin-button:focus,
vaadin-button:focus-visible {
  outline: 2px solid rgba(255, 255, 255, 0.85);
  outline-offset: 2px;
}

/* Active */
vaadin-button:active {
  transform: translateY(0);
  box-shadow:
    0 6px 16px rgba(4, 43, 92, 0.2),
    inset 0 2px 4px rgba(0, 0, 0, 0.2);
}

```

Kuva 36. Painikkeiden määrittely eri tiloihin styles.css tiedostossa.

```

/* Focus */
vaadin-text-field:focus-within::part(input-field),
vaadin-number-field:focus-within::part(input-field),
vaadin-combo-box:focus-within::part(input-field),
vaadin-text-area:focus-within::part(input-field) {
  border: 2px solid #0988bf;
  box-shadow: 0 0 0 3px rgba(59, 198, 214, 0.2);
}

```

Kuva 37. Text-, number-, comboBox ja TextArea kohtien focus tilojen määrittäminen styles.css -tiedostossa.

7 ULKOASU JA RAKENNE (SPA)

Sovellus rakennettiin alusta alkaen tehtävänannon mukaisesti yksisivuisena SPA-sovelluksena, jossa kaikki näkymät renderöidään saman päälayoutin sisään ilman turhaa sivun uudelleenlatausta.

Päälayout toteutettiin MainLayout-luokassa, joka perii AppLayout -luokan. Sovellus siis jakautuu kolmeen pääosaan: header, navigaatio (drawer) ja footer. Kaikki näkymät on liitetty tähän layoutiin käyttämällä Vaadinin reititystä seuraavasti:

```
@Route(value = "polku", layout = MainLayout.class)
```

```
@Route(value = "scooters", layout = MainLayout.class) @Menu(title = "Potku laudat", icon = "cart")
public class ScooterView extends VerticalLayout {
```

Kuva 38. Esimerkki reitityksestä ja navigaatiosta ScooterView.java tiedostosta.

Kaikki näkymät käyttävät tätä samaa yhtenäistä rakennetta ja käyttäjäkokemus pysyy johdonmukaisena koko sovelluksessa.

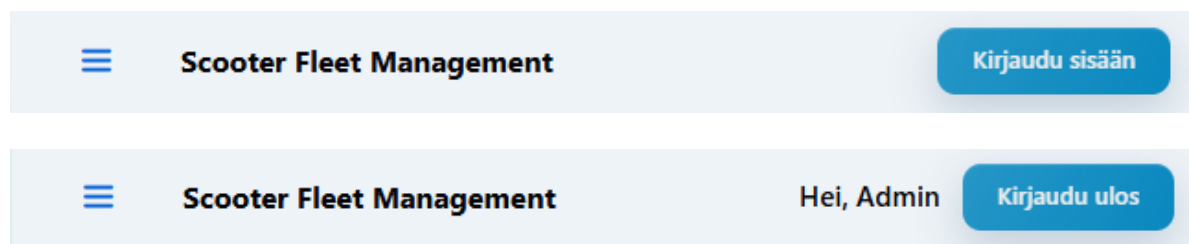
7.1 MainLayout rakenne

MainLayout sisältää sovelluksen yhteisen käyttöliittymärakenteen, joka näkyy kaikissa näkymissä. Header on sijoitettu AppLayout-komponentin navbar-alueelle, jotta se pysyy jatkuvasti näkyvissä käyttäjän siirtyessä näkymästä toiseen.

Header sisältää seuraavat osat:

- sovelluksen nimen (Scooter Fleet Management)
- kirjautuneen käyttäjän tunnistetiedon, joka näytetään navbarissa käyttäjänimenä
- autentikointiin liittyvän painikkeen, joka näkyy kirjautumattomalle käyttäjälle muodossa "Kirjaudu sisään" ja kirjautuneelle käyttäjälle "Kirjaudu ulos"
- DrawerToggle-komponentti sivunavigaation avaamista ja sulkemista varten

Tässä ratkaisussa headerin oikea reuna reagoi käyttäjän kirjautumistilaan. Kun käyttäjä ei ole kirjautunut, navbarissa näytetään sisäänkirjautumispainike. Kun käyttäjä on kirjautunut, navbarissa näytetään tervehdys, käyttäjän tunnistetieto sekä sisäänkirjautumispainike muuttuu uloskirjautumispainikkeeksi.



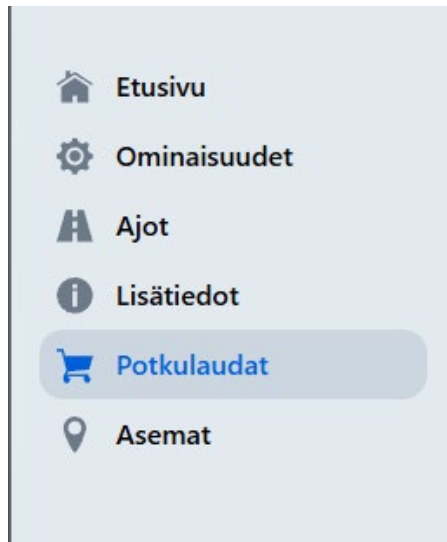
Kuvat 39. Header kirjautumattomana ja kirjautuneena. Kirjautuneena käyttäjän tunnistetieto (Admin) ja uloskirjautumispainike näkyvät navbarissa.

7.2 Navigaatio

Navigaatio tehtiin AppLayoutin drawer-osioon, jossa on linkit sovelluksen eri näkymiin.

Navigaatioissa on:

- jokaisella näkymällä oma linkki
- jokaisella linkillä oma ikoni
- aktiivinen sivu korostuu visuaalisesti



Kuva 40. Navigaatio sovelluksessa.

7.3 Footer

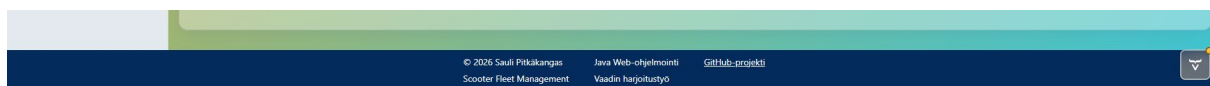
Sovelluksessa on erillinen footer-komponentti, joka sisältää:

- tekijän nimen
- copyright-tiedot
- lisätietoa sovelluksesta
- linkin GitHub-projektiin

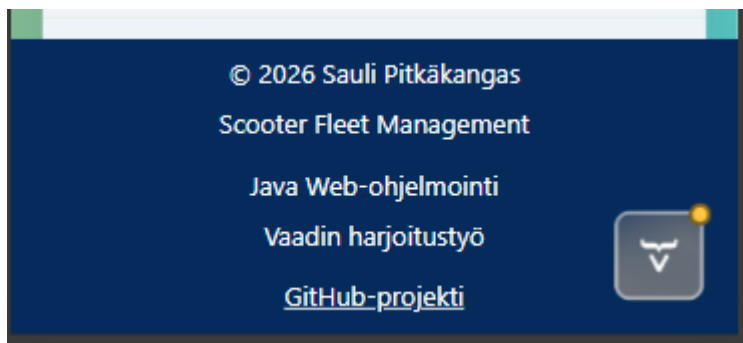
Footer on kiinnitetty näkymän alaosaan ja se erottuu visuaalisesti muusta sisällöstä esimerkiksi taustan ja layoutin avulla. Responsiivisuus on otettu huomioon eri näytöillä ja se on toteutettu styles.css -tiedostossa mediaQueryllä seuraavasti:

```
/* Responsiivisuus */
@media (max-width: 900px) {
  .footer-content {
    flex-direction: column;
    align-items: center;
    text-align: center;
    gap: 10px;
  }
  .footer-column {
    align-items: center;
    white-space: normal;
  }
}
```

Kuva 41. MediaQuery footerille styles.css tiedostossa.



Kuva 42. Footer koko näytön levyisenä (2025 x 844).



Kuva 43. Footer responsiivisesti mobiiliversiossa (390 x 844).

7.4 Kolme erityyppistä näkymää

Sovellukseen rakennettiin kolme rakenteeltaan (ja ulkoasultaan) erilaista näkymää. Kaikki näkymät hyödyntävät yhteistä MainLayout-rakennetta, joten header, navigaatio ja footer pysyvät samoina koko sovelluksessa. Itse sisältöalueet on kuitenkin tehty eri tavoilla käyttötarkoituksen ja tehtävänannon mukaan.

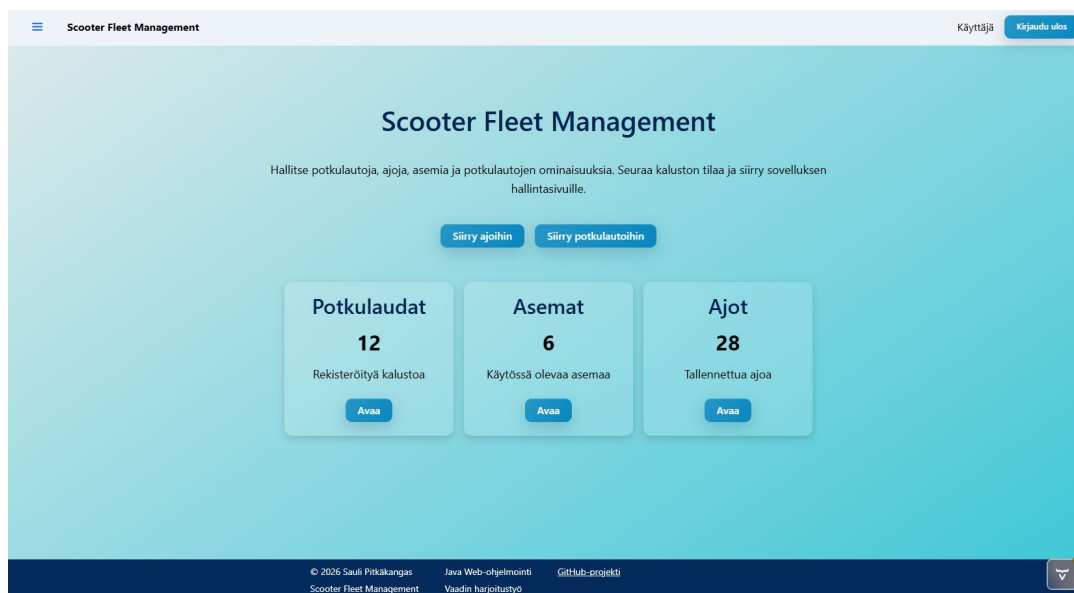
7.4.1 Etusivu – dashboard-tyylinen näkymä

Etusivu tehtiin dashboard-tyylisenä päänäkymänä, jonka tarkoitus on toimia sovelluksen yleiskatsauksena ja lähtöpisteenä eri hallintasivulle siirtymisessä.

Etusivun yläosassa on informaatioalue (hero section), jossa näytetään sovelluksen nimi, lyhyt kuvaus järjestelmän tarkoituksesta sekä painikkeet tärkeimpiin näkymiin (potkulaudat ja ajot) siirtymistä varten. Tämän alle toteutin yhteenvetokortit, joissa esitetään keskeisiä kokonaisuuksia, kuten potkulaudat, asemat ja ajot. Jokaisessa kortissa näytetään otsikko, lukuarvo tallennetuista tiedoista, lyhyt selite sekä painike, jolla käyttäjä voi siirtyä kyseiseen näkymään.

Rakenteellisesti etusivu eroaa muista näkymistä siten, että siinä ei ole taulukkoa tai lomaketta pääelementtinä, vaan se koostuu informaatio-osioista ja korteista. Visuaalisesti etusivu on muuta sovellusta enemmän dashboard-tyyppinen ja toimii yleisnäkymänä eikä varsinaisena CRUD-hallinta sivuna, kuten muut sivut.

(Huom! Tässä vaiheessa muutin sovelluksen väripalettia omaan silmään paremmaksi, joten värimaailma eroaa raportin muista yllä olevista kuvakaappauksista.)



Kuva 44. Etusivun rakenne: hero-osa, toimintapainikkeet ja yhteenvetokortit.

7.4.2 Ajot – listaus, suodatus ja muokkaus samalla näkymällä

Ajot-näkymä toteutettiin hallintanäkymänä, jossa käyttäjä voi tarkastella, hakea ja muokata ajoihin liittyviä tietoja samassa näkymässä.

Näkymän rakenne koostuu kolmesta pääosasta:

- uuden ajon lisäyspainikkeesta
- suodatusosiosta
- sisältöalueesta, jossa grid-taulukko ja lomake näkyvät rinnakkain

Suodatusosio sisältää useita hakuehtoja, kuten hakusanan, statuksen, lähtöaseman, päivämäärävalin ja lajittelun. Näiden avulla käyttäjä voi rajata gridissä näkyviä tietoja. Näistä kerrottu tarkemmin ylempänä raportissa kohdassa [5 SUODATTAMINEN JA CRITERIA API](#). Varsinainen sisältöalue tehtiin siten, että vasemmalla näkyy ajot sisältävä grid ja oikealla lisäys/muokkauslomake. Käyttäjä voi valita gridistä ajon, jolloin sen tiedot avautuvat suoraan lomakkeelle muokattavaksi ilman erillistä dialogia tai sivun vaihtoa.

Tämä näkymä eroaa etusivusta sekä rakenteellisesti että toiminnallisesti. Ajot-näkymässä painopiste on tietojen hakemisessa ja muokkaamisessa, kun taas etusivu toimii yleiskatsauksena. Visuaalisesti näkymä muistuttaa perinteistä hallintapaneelia, joissa tiedot, hakutoiminnot ja lisäys/muokkauslomake on sijoitettu samaan työtilaan.

The screenshot displays the 'Scooter Fleet Management' application interface. At the top, there's a header with a menu icon, the title 'Scooter Fleet Management', and user information 'Käyttäjä' and 'Kirjaudu ulos'. Below the header is a blue button 'Lisää uusi ajo'. The main area features a search bar 'Hakusana' with the placeholder 'Hae esim. scooter, asema...' and several filter dropdowns: 'Status' (Valitse status), 'Lähtöasema' (Valitse lähtöasema), 'Ajon aloituspäivä', 'Ajon lopetuspäivä', and 'Lajittelu' (Valitse lajittelu). There are 'Hae' and 'Tyhjennä' buttons. Below the filters is a table with 15 rows of trip data. The table columns are: ID, Aloitusaika, Lopetusaika, Kilometri määrä, Ajon hinta, Status, Potkulauta, Aloitusasema, and Pääteasema. To the right of the table is a sidebar with additional filter controls for 'Aloitusaika', 'Lopetusaika', 'Kilomerimäärä', 'Status', 'Hinta', 'Potkulauta', 'Lähtöasema', and 'Pääteasema'. The footer contains copyright information '© 2026 Sauli Pitkakangas', 'Scooter Fleet Management', 'Java Web-ohjelmointi', 'Vaadin harjoitustyö', and a 'GitHub-projekti' link.

ID	Aloitusaika	Lopetusaika	Kilometri määrä	Ajon hinta	Status	Potkulauta	Aloitusasema	Pääteasema
1	2026-03-10T...	2026-03-10T...	3.5	5.9	COMPLETED	SN1001 - Ni...	Tori	Teatteri
2	2026-03-10T...	2026-03-10T...	2.8	4.8	COMPLETED	SC-2002 - Xi...	Teatteri	Savonia
3	2026-03-11T...	2026-03-11T...	4.2	6.7	COMPLETED	SC-2003 - S...	Savonia	Tori
4	2026-03-12T...	2026-03-12T...	10.1	10.2	COMPLETED	SC-3001 - A...	Tori	Teatteri
5	2026-03-16T...	2026-03-16T...	4.0	3.4	COMPLETED	SC-1002 - Ni...	Teatteri	Savonia
6	2026-03-23T...	2026-03-23T...	10.0	5.0	CANCELLED	SN-9999 - Bi...	Teatteri	Savonia
7	2026-03-22T...	2026-03-22T...	2.0	0.6	FAILED	SC-2002 - Xi...	Kotivarasto	Teatteri
8	2026-03-18T...	2026-03-18T...	3.1	4.2	COMPLETED	SN1001 - Ni...	Tori	Teatteri
9	2026-03-24T...	2026-03-24T...	3.2	4.3	COMPLETED	SN1001 - Ni...	Tori	Teatteri
10	2026-03-12T...	2026-03-12T...	4.8	6.1	COMPLETED	SC-2002 - Xi...	S-market Kei...	Matkakeskus
11	2026-03-03T...	2026-03-03T...	3.9	8.6	COMPLETED	SC-3456 - Tier	Matkakeskus	Männistön t...
12	2026-03-07T...	2026-03-07T...	5.4	6.9	COMPLETED	SC-3001 - A...	Tori	S-market Kei...
13	2026-03-02T...	2026-03-02T...	3.5	2.9	FAILED	SC-3458 - Tier	K-market Vel...	Teatteri
14	2026-03-16T...	2026-03-16T...	3.1	1.9	CANCELLED	SC-3456 - Tier	S-market Sat...	Matkakeskus
15	2026-03-22T...	2026-03-22T...	7.5	12.0	COMPLETED	SN-9874 - Ai...	Matkakeskus	K-market De...

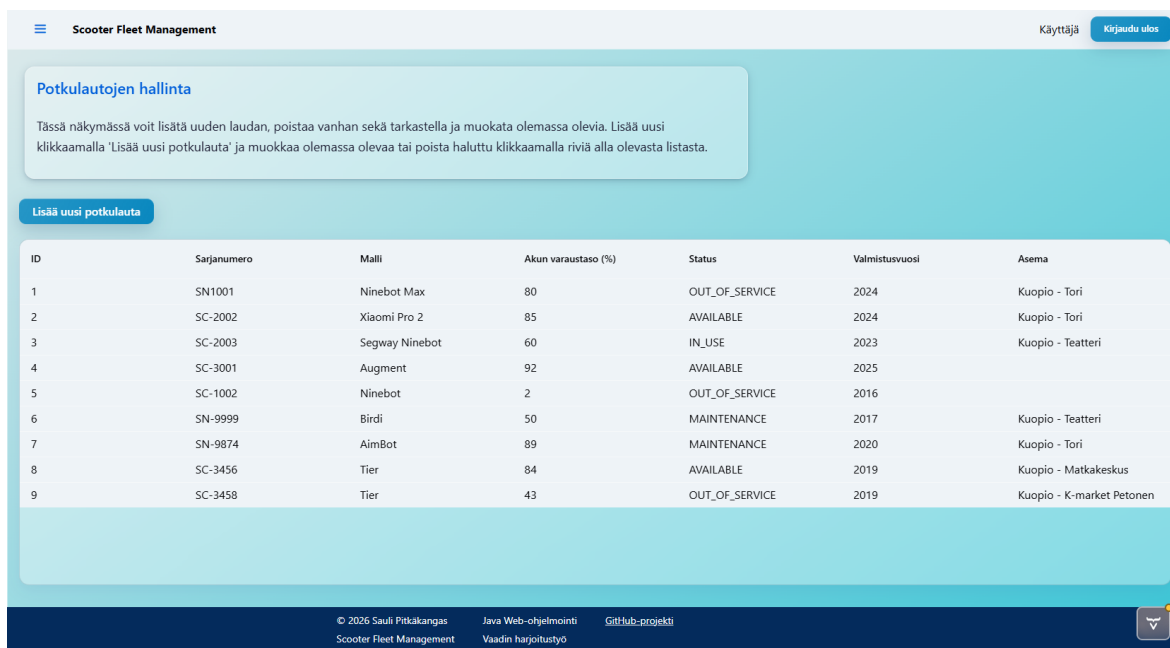
Kuva 45. Ajot-näkymä: Suodatuskentät, grid ja oikealla näkyvä lomake.

7.4.3 Potkulaudat – informaatio + grid + dialog

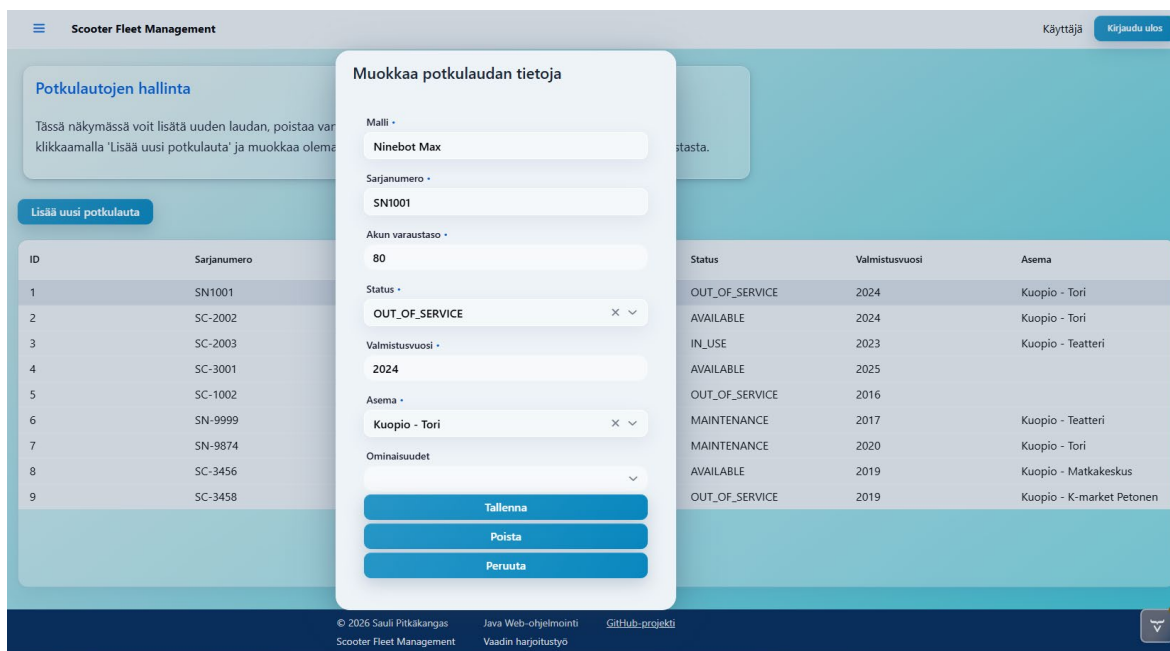
Potkulaudat-näkymä toteutettiin kolmantena rakenteeltaan erilaisena näkymänä. Sen yläosassa näytetään otsikko ja lyhyt informaatioalue, jonka alla on uuden potkulaudan lisäyspainike ja potkulautojen tiedot näyttävä taulukko.

Toisin kuin Ajot-näkymässä, potkulautojen muokkaus ja lisäys ei tapahdu gridin-tilin vieressä näkyvässä lomakkeessa, vaan erillisenä dialogi-ikkunassa. Kun käyttäjä lisää uuden potkulaudan tai valitsee olemassa olevan potkulaudan gridistä, avautuu dialogi, jonka sisällä näytetään muokkauslomake. Otsikko lomakkeessa muuttuu dynaamisesti sen mukaan, lisätäänkö uusi vai muokataanko olemassa olevaa. Tämä ratkaisu tekee näkymästä rakenteellisesti erilaisen verrattuna Ajot-näkymään, jossa lomake on jatkuvasti näkyvässä osana sivun layouttia.

Visuaalisesti Potkulaudat-näkymä eroaa muista näkymistä siten, että siinä yhdistyvät informaatioalue, taulukkomuotoinen listaus ja modaalinen muokkausikkuna.



Kuva 46. Potkulaudat-näkymä: otsikko- ja infoalue sekä grid.



Kuva 47. Potkulautojen muokkaus dialogi-ikkunassa.

7.4.4 Yhteenveto näkymien eroista

Kolmen näkymän toteutuksessa pyrin siihen, että jokaisella sivulla on oma käyttötarkoituksensa ja sen mukainen rakenne.

1. ETUSIVU toimii dashboard-tyyppisenä yleisnäkymänä
2. AJOT-sivu toimii hakua, listausta ja samassa näkymässä tapahtuvaa muokkausta tukevana hallintanäkymänä.
3. POTKULAUDAT-sivu toimii listausnäkymänä, jossa muokkaus ja lisäys on siirretty erilliseen dialogi-ikkunaan.

Näin tehtävänannon vaatimus vähintään kolmesta erityyppisestä näkymästä täyttyy rakenteellisesti ja visuaalisesti. Näiden kolmen lisäksi on vielä sivut: 'Ominaisuudet', 'Lisätiedot' ja 'Asemat', jotka kaikki käyttävät täysin samaa rakennetta.

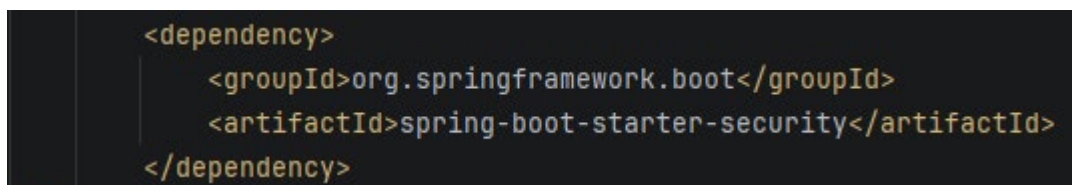
8 AUTENTIKOINTI JA TIETOTURVA

Sovellukseen toteutettiin käyttäjien autentikointi ja roolipohjainen käyttöoikeuksien hallinta hyödyntäen Spring Securityä ja Vaadinin integraatiota. Tietoturvaratkaisun avulla käyttäjät voivat kirjautua sovellukseen, rekisteröityä uutena käyttäjänä ja käyttää eri näkymiä omien oikeuksien mukaisesti.

Lisäksi sovelluksessa on käyttäjäkohtaisia toiminnallisuuksia, kuten profiilikuvan lisääminen sekä kustomoitu virhesivu tilanteisiin, joissa käyttäjällä ei ole riittäviä käyttöoikeuksia.

8.1 Spring Securityn käyttöönotto

Spring Security otettiin käyttöön lisäämällä projektiin riippuvuus spring-boot-starter-security (pom.xml), jonka jälkeen sovellus siirtyi automaattisesti suojattuun tilaan, jossa kaikki näkymät vaativat autentikoinnin.



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Kuva 48. Spring Security -riippuvuuden lisääminen pom.xml-tiedostoon

Tietoturvan konfigurointi toteutettiin SecurityConfig-luokassa, jossa määriteltiin sovelluksen suojausasetukset sekä kirjautumisnäkymäksi LoginView.

Salasanojen käsittely tehtiin käyttäen BCryptPasswordEncoder-luokkaa, joka tallentaa salasanat hash-muodossa tietokantaan

Sovelluksen käynnistyksen yhteydessä luodaan oletuskäyttäjät (Admin, Super ja User) DataInitializer-luokan avulla.

Valmiiksi luotujen tilien käyttäjätunnukset ja salasanat ovat:

- User – user123
- Super – super123
- Admin – admin123


```

@EnableWebSecurity @SaleSan
@Configuration
public class SecurityConfig {

    // Määrittää sovelluksen pääsuojasasetukset ja login-näkymän
    @Bean @SaleSan
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
            .requestMatchers(🔒"/images/**", 🔒"/profile-images/**").permitAll()
        );

        http.with(VaadinSecurityConfigurer.vaadin(), VaadinSecurityConfigurer config -> {
            config.loginView(LoginView.class);
        });

        return http.build();
    }

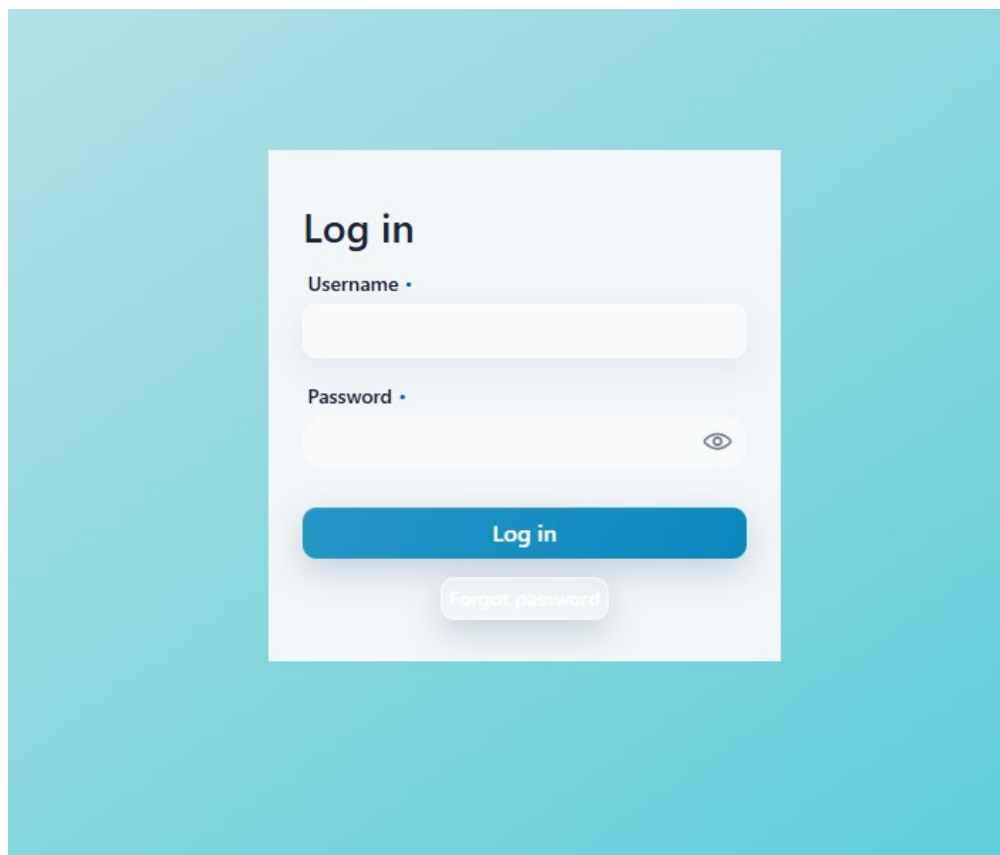
    // Sallii H2-consolin käytön ilman autentikointia
    @Bean @SaleSan
    @Order(Ordered.HIGHEST_PRECEDENCE)
    public SecurityFilterChain h2ConsoleSecurityFilterChain(HttpSecurity http) throws Exception {
        http.securityMatcher(PathRequest.toH2Console())
            .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth.anyRequest().permitAll())
            .csrf(AbstractHttpConfigurer::disable)
            .headers( HeadersConfigurer<HttpSecurity> headers ->
                headers.frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)
            );

        return http.build();
    }

    // Salasanan hashaus (BCrypt)
    @Bean @SaleSan
    public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}

```

Kuva 49. SecurityConfig.java



Kuva 50. Kirjautumisnäkyminen osoitteessa: /login.

```

@Configuration & SaleSan
public class DataInitializer {

    // Alustaa testikäyttäjät (Admin, Super, User)
    @Bean & SaleSan
    CommandLineRunner initUsers(AppUserRepository appUserRepository, PasswordEncoder passwordEncoder) {
        return String[] args -> {
            if (!appUserRepository.existsByUsername("Admin")) {
                AppUser admin = new AppUser();
                admin.setUsername("Admin");
                admin.setPasswordHash(passwordEncoder.encode(rawPassword: "admin123"));
                admin.setRoles(Set.of(Role.ADMIN));
                appUserRepository.save(admin);
            }

            if (!appUserRepository.existsByUsername("Super")) {
                AppUser superUser = new AppUser();
                superUser.setUsername("Super");
                superUser.setPasswordHash(passwordEncoder.encode(rawPassword: "super123"));
                superUser.setRoles(Set.of(Role.SUPER));
                appUserRepository.save(superUser);
            }

            if (!appUserRepository.existsByUsername("User")) {
                AppUser user = new AppUser();
                user.setUsername("User");
                user.setPasswordHash(passwordEncoder.encode(rawPassword: "user123"));
                user.setRoles(Set.of(Role.USER));
                appUserRepository.save(user);
            }
        };
    }
}

```

Kuva 51. Toteutus käyttäjien alustamiseen DataInitializer.java tiedostossa.

8.2 Käyttäjä entiteetin luominen

Sovelluksessa toimii käyttäjäentiteetti AppUser, joka määrittelee käyttäjään liittyvät tiedot ja niiden tallennuksen tietokantaan. Entiteetti sisältää käyttäjänimen, salasanan hash-muodossa sekä käyttäjän roolit.

Salasanaa ei tallenneta selkokielisenä, vaan se muunnetaan BCryptPasswordEncoder-luokan avulla hash-muotoon ennen tallennusta. Tämä parantaa tietoturvaa ja estää salasanojen suoran lukemisen tietokannasta.

SELECT * FROM APP_USER;

ID	PASSWORD_HASH	PROFILE_IMAGE_PATH	USERNAME
9	\$2a\$10\$Z8DFqT304oHGikXGWcaLy.ZGuWQek4a4LLKdr4uF2EAZo0EGGDI.W	/images/user-9.png	Admin
10	\$2a\$10\$IVCjiteoEIlk0sJgQYHw0.kfvNgbHJ/.KfBZxiJrUyYUQDoEoLDLa	null	Super
11	\$2a\$10\$9Fc1OahF3X03/SBcUd6D2eBotBDsC4lkUgh9Stx.AALf5uEjbn.q	/profile-images/user-11.png	User
12	\$2a\$10\$xtLcCZrGJ6/NvE6PTYC69u5xNGqrbq3OdWAOUMPR384B.DhnGAixe	null	Testi1

(4 rows, 2 ms)

Kuva 52. Hashauksen todennus H2-consolista.

Käyttäjän roolit toteutettiin erillisellä Role-enumilla, joka sisältää arvot ADMIN, SUPER ja USER. Käyttäjällä voi olla yksi tai useampi rooli, ja roolit tallennetaan tietokantaan kokoelmana.

Käyttäjien autentikointi toteutettiin AppUserService-luokan avulla, jossa käyttäjä haetaan tietokannasta ja muunnetaan Spring Securityn käyttämään UserDetails-muotoon, mahdollistaen käyttäjän tunnistamisen ja roolien hyödyntämisen käyttöoikeuksien hallinnassa.

```

@Entity @SaleSan
@Table(name = "app_user")
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true) 2 usages
    private String username;

    @Column(nullable = false) 2 usages
    private String passwordHash;

    @ElementCollection(fetch = FetchType.EAGER) 2 usages
    @Enumerated(EnumType.STRING)
    @CollectionTable(name = "app_user_roles", joinColumns = @JoinColumn(name = "user_id"))
    @Column(name = "role")
    private Set<Role> roles = new HashSet<>();

    private String profileImagePath; 2 usages

    public AppUser() { @SaleSan
    }

    public Long getId() { return id; }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

```

Kuva 53. Käyttäjäentiteetin määrittely AppUser-luokassa.

```

@Service @SaleSan
public class AppUserDetailsService implements UserDetailsService {

    private final AppUserRepository userRepository; 2 usages

    // --- Konstruktor ---
    public AppUserDetailsService(AppUserRepository userRepository) { this.userRepository = userRepository; }

    // Lataa käyttäjän ja sen roolit autentikointia varten
    @Override no usages @SaleSan
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        AppUser user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("Käyttäjää ei löytynyt: " + username));

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPasswordHash(),
            user.getRoles().stream() Stream<Role>
                .map( Role role -> new SimpleGrantedAuthority("ROLE_" + role.name())) Stream<SimpleGrantedAuthority>
                .collect(Collectors.toSet())
        );
    }
}

```

Kuva 54. Käyttäjän haku ja muuntaminen Spring Securityn muotoon AppUserService-luokassa.

```
// Luo uuden käyttäjän ja tallentaa sen tietokantaan
public void registerUser(String username, String password) { 1 usage 2 SaleSan
    if (userRepository.existsByUsername(username)) {
        throw new IllegalArgumentException("Käyttäjätunnus on jo käytössä");
    }

    AppUser user = new AppUser();
    user.setUsername(username);
    user.setPasswordHash(passwordEncoder.encode(password));
    user.setRoles(Set.of(Role.USER));
    userRepository.save(user);
}

// Hakee tällä hetkellä kirjautuneen käyttäjän
public AppUser getCurrentUser() { 3 usages 2 SaleSan
    String username = SecurityContextHolder.getContext().getAuthentication().getName();
    return userRepository.findByUsername(username)
        .orElseThrow(() -> new IllegalArgumentException("Kirjautunutta käyttäjää ei löytynyt"));
}

// Tallentaa käyttäjän profiilikuvan polun
public void saveProfileImagePath(String imagePath) { 1 usage 2 SaleSan
    AppUser user = getCurrentUser();
    user.setProfileImagePath(imagePath);
    userRepository.save(user);
}
```

Kuva 55. Salasanan hash-muodossa tallentaminen rekisteröinnin yhteydessä UserService-luokassa.

8.3 Roolikohtaiset sivunäkymät

Sovelluksessa on roolipohjainen käyttöoikeuksien hallinta, joka saatiin hyödyntämällä Spring Securityn tarjoamia annotaatioita. Näkymien käyttöoikeudet määriteltiin suoraan view-luokissa annotaatioiden avulla.

Kaikille käyttäjille avoimet sivut, kuten etusivu (/), kirjautumissivu (/login), ja rekisteröitymissivu (/register), merkittiin annotaatiolla `@AnonymousAllowed`, jolloin niitä voidaan käyttää ilman kirjautumista.

Kirjautuneille käyttäjille sallittiin pääsy tiettyihin näkymiin käyttämällä `@PermitAll`. Tällainen näkymä on esimerkiksi ajot (/rides), joka on kaikkien kirjautuneiden käyttäjien käytettävissä.

Roolikohtaiset käyttöoikeudet toteutettiin `@RolesAllowed` annotaatiolla, jonka avulla näkymät rajattiin tiettyjen roolien käyttäjille. Esimerkiksi potkulaudat (/scooters), lisätiedot (/scooterdetails) ja asemat (/stations) ovat käytettävissä vain rooleille SUPER ja USER. Ominaisuudet-sivu (/features) on puolestaan rajattu vain ADMIN käyttäjälle.

Käyttäjät näkevät ja voivat käyttää vain niitä näkymiä, joihin heidän roolinsa antaa oikeudet.

Alla koostelista (korvaamaan useat kuvakaappaukset jokaiselta sivulta), miten roolikohtaiset näkymät ovat toteutettu:

Näky kirjautumattomille ja kirjautuneille:

- MainLayout.java (navbar, sidebar ja footer)
- HomeView.java (etusivu)
- LoginView.java (kirjautumissivu)
- RegisterView.java (rekisteröitymissivu)
 - / -> `@AnonymousAllowed`

- /login -> @AnonymousAllowed
- /register -> @AnonymousAllowed

Näkyvyy kirjautuneille:

- RideView.java (ajot-näkymä)
 - /rides -> @PermitAll

Näkyvyy vain käyttäjille, joilla ovat roolit SUPER ja USER:

- ScooterView.java (potkulaudat-näkymä)
- ScooterDetailView.java (lisätiedot-näkymä)
- StationView.java (asemat-näkymä)
 - /scooters -> @RolesAllowed({ "SUPER", "USER" })
 - /scooterdetails -> @RolesAllowed({ "SUPER", "USER" })
 - /stations -> @RolesAllowed({ "SUPER", "USER" })

Näkyvyy vain ADMIN käyttäjälle:

- FeatureView.java (ominaisuudet-näkymä)
 - /features -> @RolesAllowed({ "ADMIN" })

8.4 Rekisteröitymissivu

Rekisteröitymissivun (/register) avulla, uusi käyttäjä voi luoda itselleen tunnukset. Rekisteröityminen on sallittu ilman kirjautumista ja näkymä on merkitty annotaatiolla @AnonymousAllowed.

Sivulla käyttäjä syöttää käyttäjänimen sekä salasanan kahteen kertaan. Sovellus tarkistaa, että kaikki kentät on täytetty ja että salasanat täsmäävät ennen käyttäjätilin luomista.

Käyttäjän tallennus toteutetaan UserService-luokan avulla, jossa salasana muunnetaan hash-muotoon ennen tietokantaan tallentamista. Lisäksi tarkistetaan, ettei käyttäjänimi ole jo käytössä.

Onnistuneen rekisteröitymisen jälkeen käyttäjälle näytetään ilmoitus ja hänet voidaan ohjata kirjautumissivulle.

```
// Rekisteröinti näkymä uusille käyttäjille
public RegisterView(UserService userService) { @SaleSan

    TextField username = new TextField(label: "Käyttäjätunnus");
    PasswordField password = new PasswordField(label: "Salasana");
    PasswordField confirmPassword = new PasswordField(label: "Vahvista salasana");

    Button registerBtn = new Button(text: "Rekisteröidy", ClickEvent<Button> e -> {

        // Tarkistaa syötteen ja luo käyttäjän
        if (username.isEmpty() || password.isEmpty() || confirmPassword.isEmpty()) {
            Notification.show(text: "Täytä kaikki kentät");
            return;
        }

        // Varmistaa salasanat
        if (!password.getValue().equals(confirmPassword.getValue())) {
            Notification.show(text: "Salasanat eivät täsmää");
            return;
        }

        try {
            userService.registerUser(
                username.getValue(),
                password.getValue()
            );
            Notification.show(text: "Rekisteröinti onnistui");
            getUI().ifPresent(ui -> ui.navigate(location: "login"));
        } catch (IllegalArgumentException ex) {
            Notification.show(ex.getMessage());
        }
    });

    add(username, password, confirmPassword, registerBtn);
    setAlignItems(Alignment.CENTER);
}
```

Kuva 56. Rekisteröitymissivun käyttöliittymä ja validointi RegisterView-luokassa.

```
// Luo uuden käyttäjän ja tallentaa sen tietokantaan
public void registerUser(String username, String password) { 1 usage @SaleSan
    if (userRepository.existsByUsername(username)) {
        throw new IllegalArgumentException("Käyttäjätunnus on jo käytössä");
    }

    AppUser user = new AppUser();
    user.setUsername(username);
    user.setPasswordHash(passwordEncoder.encode(password));
    user.setRoles(Set.of(Role.USER));
    userRepository.save(user);
}

// Hakee tällä hetkellä kirjautuneen käyttäjän
public AppUser getCurrentUser() { 3 usages @SaleSan
    String username = SecurityContextHolder.getContext().getAuthentication().getName();
    return userRepository.findByUsername(username)
        .orElseThrow(() -> new IllegalArgumentException("Kirjautunutta käyttäjää ei löytynyt"));
}

// Tallentaa käyttäjän profiilikuvan polun
public void saveProfileImagePath(String imagePath) { 1 usage @SaleSan
    AppUser user = getCurrentUser();
    user.setProfileImagePath(imagePath);
    userRepository.save(user);
}
```

Kuva 57. Uuden käyttäjän luonti ja salasanan hash-muotoon muuntaminen UserService-luokassa.

Kuva 58. Sovelluksen yksinkertainen rekisteröitymissivu.

8.5 Kustomoitu virheviesti

Kustomoitu virhesivu luotiin tilanteisiin, joissa käyttäjällä ei ole riittäviä käyttöoikeuksia avata tiettyä näkymää. Oletuksena Vaadin näyttää geneerisen virheilmoituksen, mutta tässä toteutuksessa se korvattiin omalla näkymällä.

Kustomoitu virhesivu tehtiin CustomAccessDeniedError-luokan avulla, joka implementoi HasError-Parameter<AccessDeniedException>-rajapinnan, mahdollistaen käyttöoikeusvirheiden käsittelyn ja oman näkymän näyttämisen käyttäjälle virhetilanteissa.

Sivu näyttää käyttäjälle selkeän ilmoituksen siitä, että hänellä ei ole oikeuksia kyseiseen näkymään, sekä tarjoaa mahdollisuuden siirtyä takaisin etusivulle painikkeen avulla.

Lisäksi FeatureView-näkymään lisättiin @AccessDeniedErrorRouter-annotaatio, joka ohjaa käyttöoikeusvirheen automaattisesti kustomoituun virhesivuun.

Tämän ansiosta käyttäjä saa selkeän ja käyttäjystävällisen palautteen tilanteessa, jossa käyttöoikeudet eivät riitä.

```
// Customoitu näkymä käyttöoikeusvirheelle (Access Denied)
@PermitAll no usages 👤 SaleSan
public class CustomAccessDeniedError extends VerticalLayout
    implements HasErrorParameter<AccessDeniedException> {

    // --- Konstruktor ---
    public CustomAccessDeniedError() { no usages 👤 SaleSan
        setSizeFull();
        setAlignItems(Alignment.CENTER);
        setJustifyContentMode(JustifyContentMode.CENTER);
        setSpacing(true);
    }

    // Näyttää virheviestin ja ohjaa takaisin etusivulle
    @Override no usages 👤 SaleSan
    public int setErrorParameter(BeforeEnterEvent event,
        ErrorParameter<AccessDeniedException> parameter) {

        removeAll();

        H2 title = new H2(text: "Pääsy estetty");
        Paragraph message = new Paragraph(text: "Sinulla ei ole oikeuksia avata tätä sivua.");

        Button backButton = new Button(text: "Takaisin etusivulle", ClickEvent<Button> e ->
            event.getUI().navigate(location: ""));

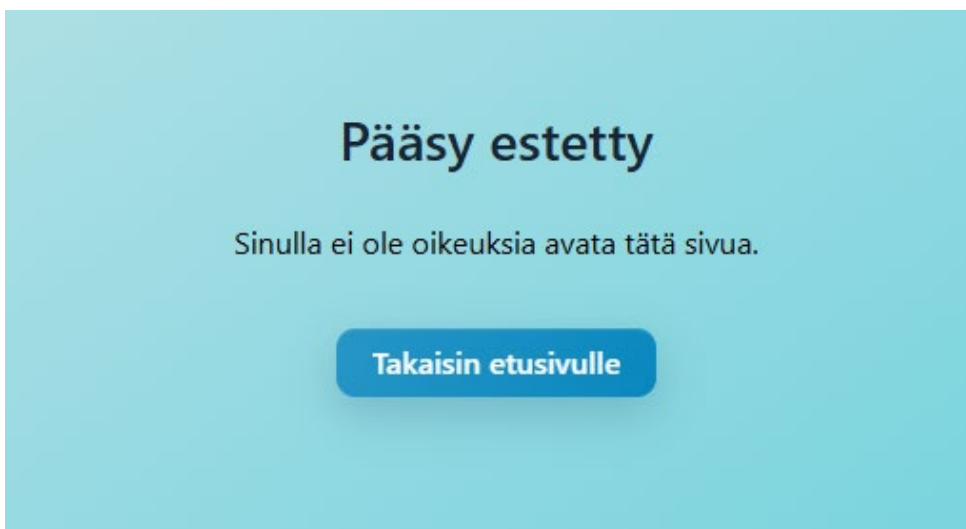
        add(title, message, backButton);

        return HttpServletResponse.SC_FORBIDDEN;
    }
}
```

Kuva 59. Kustomoidun virhesivun toteutus käyttöoikeusvirheille CustomAccessDeniedError-luokassa.

```
@Route(value = "features", layout = MainLayout.class) 👤 SaleSan
@Menu(title = "Ominaisuudet", icon = "cog")
@RolesAllowed( { "ADMIN" } )
@AccessDeniedErrorRouter
public class FeatureView extends VerticalLayout {
```

Kuva 60. Näkymän rajaaminen ADMIN-roolille ja virheohjaus FeatureView.java tiedostossa.



Kuva 61. Kustomoitu virhesivu käyttäjälle, jolla ei ole tarvittavia oikeuksia. Tässä tilanteessa on kirjautettu käyttäjällä User, ja yritetty navigoida osoitteeseen /features, jonka näkee vain ADMIN.

8.6 Oman kuvan lisääminen

Käyttäjällä on mahdollisuus lisätä oma profiilikuva. Tämä toiminnallisuus sijoitettiin erilliselle Profile-View-sivulle, joka on kirjautuneiden käyttäjien käytettävissä.

Profiilisivulla käyttäjä voi ladata kuvatiedoston Upload-komponentin avulla. Hyväksytyt tiedostotyypit rajattiin kuvatiedostoihin (png, jpeg, jpg ja gif), ja käyttäjä voi ladata vain yhden tiedoston kerrallaan. Ladattu kuva tallennetaan palvelimen tiedostojärjestelmään kansioon uploaded-profile-images.

Tallennuksen yhteydessä kuvalle muodostetaan käyttäjäkohtainen tiedostonimi, joka perustuu kirjautuneen käyttäjän tunnisteeseen. Tämän jälkeen kuvan URL tallennetaan käyttäjän profileImage-Path-kenttään tietokantaan. Kun käyttäjä avaa profiilisivun myöhemmin uudelleen, aiemmin tallennettu kuva haetaan ja näytetään sivulla automaattisesti.

Kuvien näyttäminen selaimessa mahdollistettiin ImageResourceConfig-luokan avulla, jossa määriteltiin erillinen resurssipolku profiilikuvalle. Lisäksi SecurityConfig-luokassa sallittiin /profile-images/**-polku ilman erillistä kirjautumista, jotta kuva voidaan näyttää käyttöliittymässä.

```
// Luo upload-komponentin ja käsittelee kuvan tallennuksen ja näyttämisen
public ProfileView(UserService userService) {
    this.userService = userService;

    setAlignItems(Alignment.CENTER);
    setSpacing(true);
    H2 title = new H2(text: "Oma Profiili");

    Upload upload = new Upload();
    MemoryBuffer buffer = new MemoryBuffer();
    upload.setReceiver(buffer);
    upload.setAcceptedFileTypes("image/png", "image/jpeg", "image/jpg", "image/gif");
    upload.setMaxFiles(1);

    profileImage.setAlt("Profiilikuva");
    profileImage.setWidth("200px");
    profileImage.setHeight("200px");
    profileImage.getStyle().set("border-radius", "50%");
    profileImage.getStyle().set("object-fit", "cover");
    profileImage.setVisible(false);

    loadExistingProfileImage();

    // Tallentaa ladatun kuvan tiedostoon ja päivittää käyttäjän profiilikuvan
    upload.addSucceededListener(SucceededEvent event -> {
        try {
            AppUser user = userService.getCurrentUser();

            String originalFileName = event.getFileName();
            String extension = getFileExtension(originalFileName);
            String savedFileName = "user-" + user.getId() + extension;

            Path uploadDir = Path.of(first: "uploaded-profile-images");
            Files.createDirectories(uploadDir);
            Path filePath = uploadDir.resolve(savedFileName);

            try (InputStream inputStream = buffer.getInputStream()) {
                Files.copy(inputStream, filePath, StandardCopyOption.REPLACE_EXISTING);
            }
        }
    });
}
```

Kuva 62. Profiilisivun toteutus ProfileView-luokassa, jossa käyttäjä voi ladata ja nähdä oman profiilikuvansa 1/2.

```

        } catch (IOException e) {
            Notification.show(text: "Kuvan tallennus epäonnistui");
        }
    });

    upload.addFileRejectedListener( FileRejectedEvent event ->
        Notification.show(text: "Vain kuvatiedostot ovat sallittuja"));
    add(title, profileImage, upload);
}

// Lataa ja näyttää aiemmin tallennetun kuvan
private void loadExistingProfileImage() { 1 usage  @ SaleSan
    AppUser user = userService.getCurrentUser();

    if (user.getProfileImagePath() != null && !user.getProfileImagePath().isBlank()) {
        profileImage.setSrc(user.getProfileImagePath());
        profileImage.setVisible(true);
    }
}

// Palauttaa kuvatiedoston tiedostopäätteen
private String getFileExtension(String fileName) { 1 usage  @ SaleSan
    int dotIndex = fileName.lastIndexOf(str: ".");

    if (dotIndex == -1) {
        return ".png";
    }

    return fileName.substring(dotIndex);
}

```

Kuvat 63. Profiilisivun toteutus ProfileView-luokassa, jossa käyttäjä voi ladata ja nähdä oman profiilikuvansa 2/2.

```

// Hakee tällä hetkellä kirjautuneen käyttäjän
public AppUser getCurrentUser() { 3 usages  @ SaleSan
    String username = SecurityContextHolder.getContext().getAuthentication().getName();
    return userRepository.findByUsername(username)
        .orElseThrow(() -> new IllegalArgumentException("Kirjautunutta käyttäjää ei löytynyt"));
}

// Tallentaa käyttäjän profiilikuvan polun
public void saveProfileImagePath(String imagePath) { 1 usage  @ SaleSan
    AppUser user = getCurrentUser();
    user.setProfileImagePath(imagePath);
    userRepository.save(user);
}

```

Kuva 64. Profiilikuvan polun tallentaminen käyttäjälle UserService-luokassa.

```

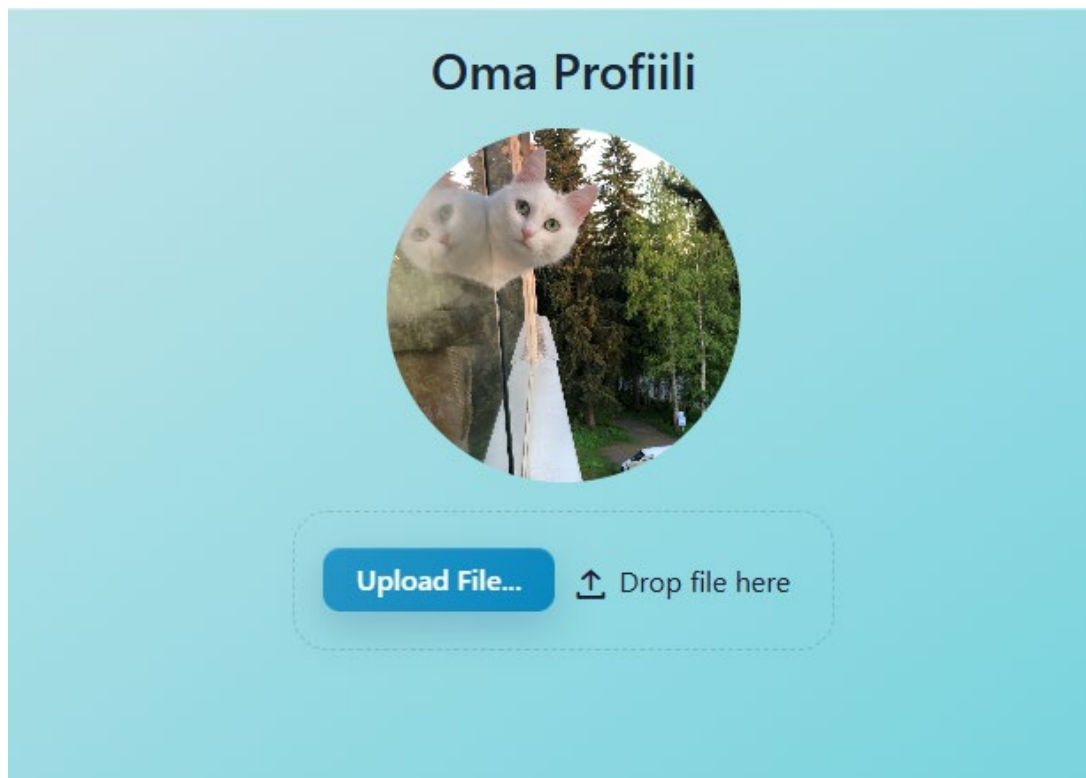
@Configuration  @ SaleSan
public class ImageResourceConfig implements WebMvcConfigurer {

    // Määrittää polun, josta ladatut kuvat haetaan
    @Override no usages  @ SaleSan
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        String uploadPath = Path.of( first: "uploaded-profile-images").toFile().getAbsolutePath();

        registry.addResourceHandler( ...pathPatterns: "/profile-images/**")
            .addResourceLocations("file:" + uploadPath + "/");
    }
}

```

Kuva 65. Profiilikuvien resurssipolun määrittely ImageResourceConfig-luokassa.



Kuva 66. Profiilikuvan näyttäminen käyttöliittymässä.

9 MUUT TOIMINNALLISUUDET

9.1 GitHub-julkaisu

Sovelluksen lähdekoodi julkaistiin GitHub-versionhallintapalveluun kurssin vaatimusten mukaisesti. Projektin lähdekooditiedostot, resurssit sekä dokumentaatio tallennettiin Git-repoon, mikä mahdollistaa työn versionhallinnan, muutosten seurannan sekä projektin jakamisen ulkopuolisille käyttäjille.

GitHub-julkaisu helpottaa myös sovelluksen jatkokehitystä ja ylläpitoa, koska kaikki muutokset on tallennettu commit-historiaan. Työ on saatavilla tarkasteltavaksi julkisesta GitHub-repositoriosta, osoitteesta: <https://github.com/Cyp6ty/vaadin-scooter-app>.

9.2 Lokalisointi

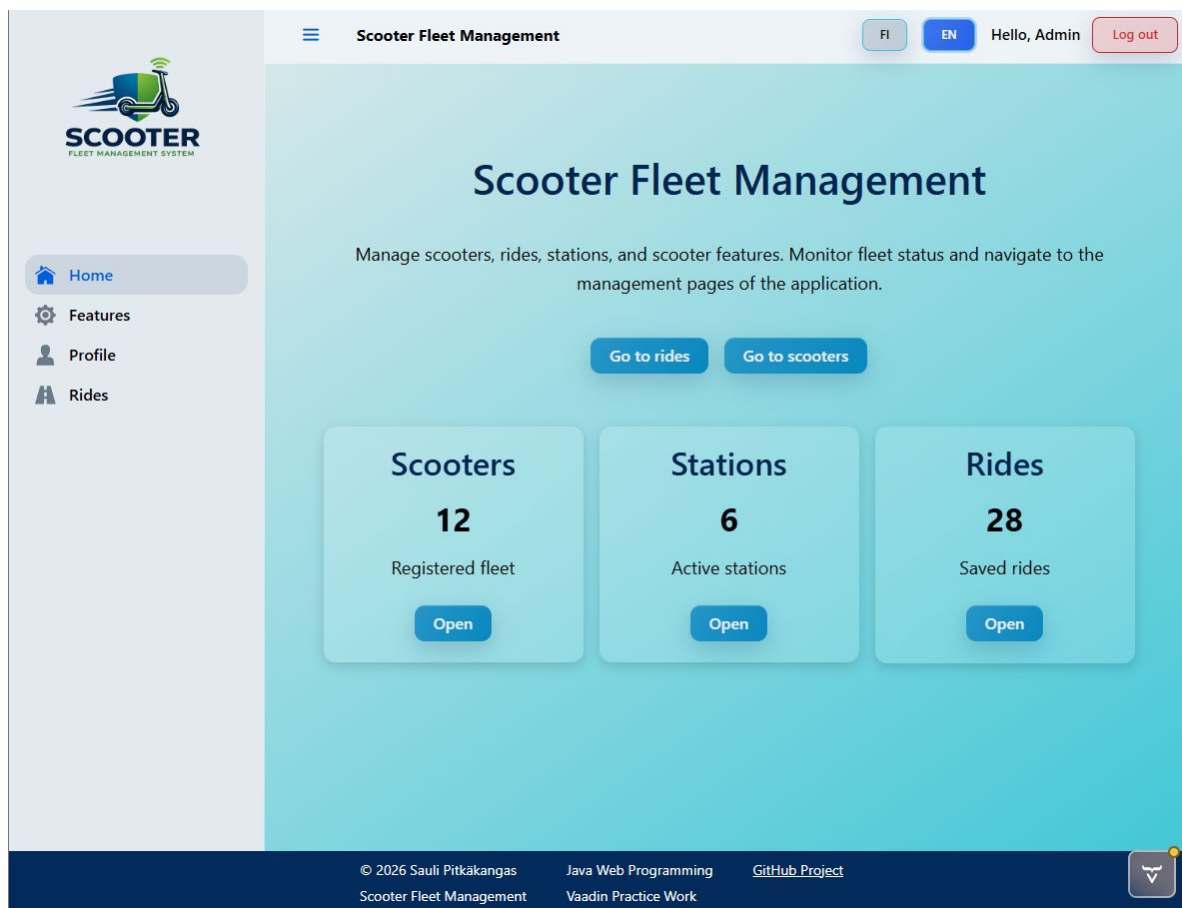
Sovellukseen tehtiin lokalisointi etusivulle, yläpalkkiin, sivunavigaatioon ja footeriin. Lisäksi kielenvaihto napit yläpalkkiin. Projektin resources-kansioon lisättiin käännöstiedostot `messages_fi.properties` ja `messages_en.properties`, joihin määriteltiin käyttöliittymän tekstit molemmille kielille. Lisäksi tehtiin oma `I18NProvider`, jonka avulla Vaadin hakee oikeat käännökset aktiivisen kielen perusteella.

Kielen vaihtoa varten yläpalkkiin lisättiin FI / EN -kielivalitsin. Käyttäjän valitsema kieli tallennetaan sessioon, jolloin valinta säilyy sivun uudelleenlatauksen jälkeen. Aktiivinen kieli korostetaan käyttöliittymässä visuaalisesti.

Lokalisointi toteutettiin näihin käyttöliittymän osiin:

- Etusivu:
 - otsikot
 - painikkeet
 - infotekstit
- Yläpalkki:
 - tervehdys
 - kirjautumis- ja uloskirjautumispainikkeet
- Sivunavigaatio:
 - Etusivu
 - Profili
 - Ominaisuudet
 - Potkulaudat
 - Ajot
 - Lisätiedot
 - Asemat
- Footer:
 - kurssitekstit
 - GitHub-linkki

Ratkaisussa käyttöliittymän kovakoodatut tekstit korvattiin `getTranslation("avain")` -kutsuilla, jolloin samat näkymät toimivat automaattisesti molemmilla kielillä.



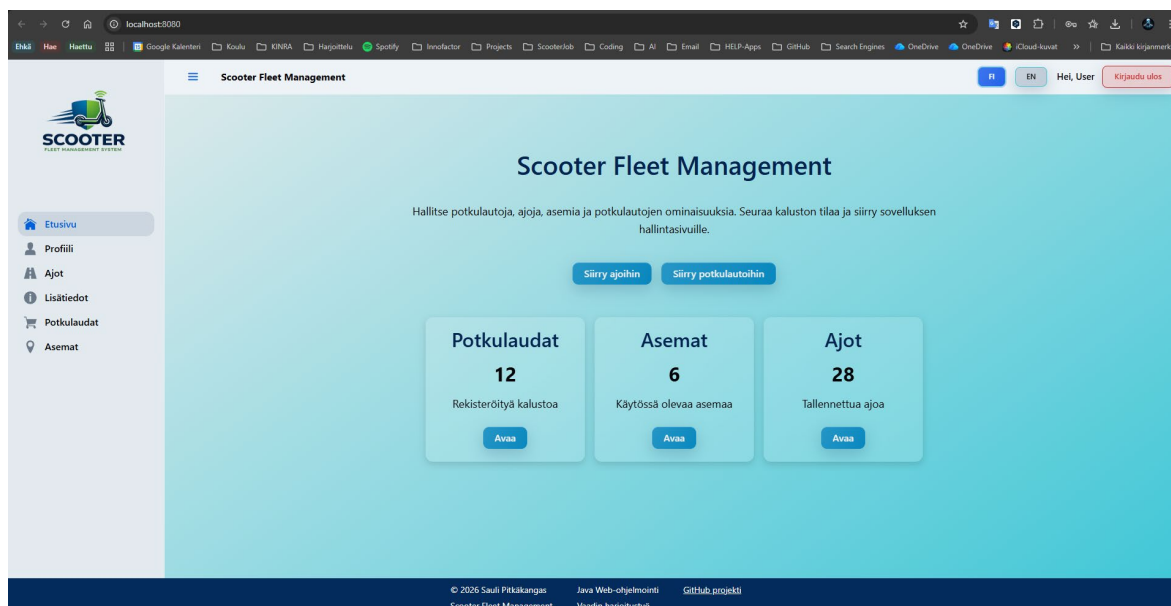
Kuva 67. Kielivalitsin yläpalkissa sekä englanniksi vaihtunut käyttöliittymä.

9.3 Docker-image ja Dockerfile

Sovellukselle on tehty toimiva Dockerfile, joka sijaitsee projektin juuressa ja on julkaistu GitHub-repositoriossa. Dockerfile rakentaa sovelluksen Mavenilla ja luo siitä ajettavan Docker-imagen. Rakennus testattiin paikallisesti komennolla `docker build -t scooter-app .`, ja image muodostui onnistuneesti.

Dockerfileä muokattiin projektin Java-version mukaiseksi, jotta kontissa käytettävä ympäristö vastaa sovelluksen asetuksia. Lisäksi build-komennossa ohitettiin testit Docker-rakennuksen yhteydessä, koska projektipohjassa ollut vanha testitiedosto esti imagen rakentamisen. Tämän jälkeen Docker-image saatiin rakennettua onnistuneesti.

Docker-kontissa sovellus käynnistyi onnistuneesti puhtaassa ympäristössä. Kontti käyttää oletuksena omaa tiedostojärjestelmää, minkä vuoksi paikallinen kehitysaineisto (tallennettu data) ei siirry automaattisesti mukaan.



Kuva 70. Sovelluksen käynnistyminen Docker-kontissa selaimessa.

9.4 Docker-Compose

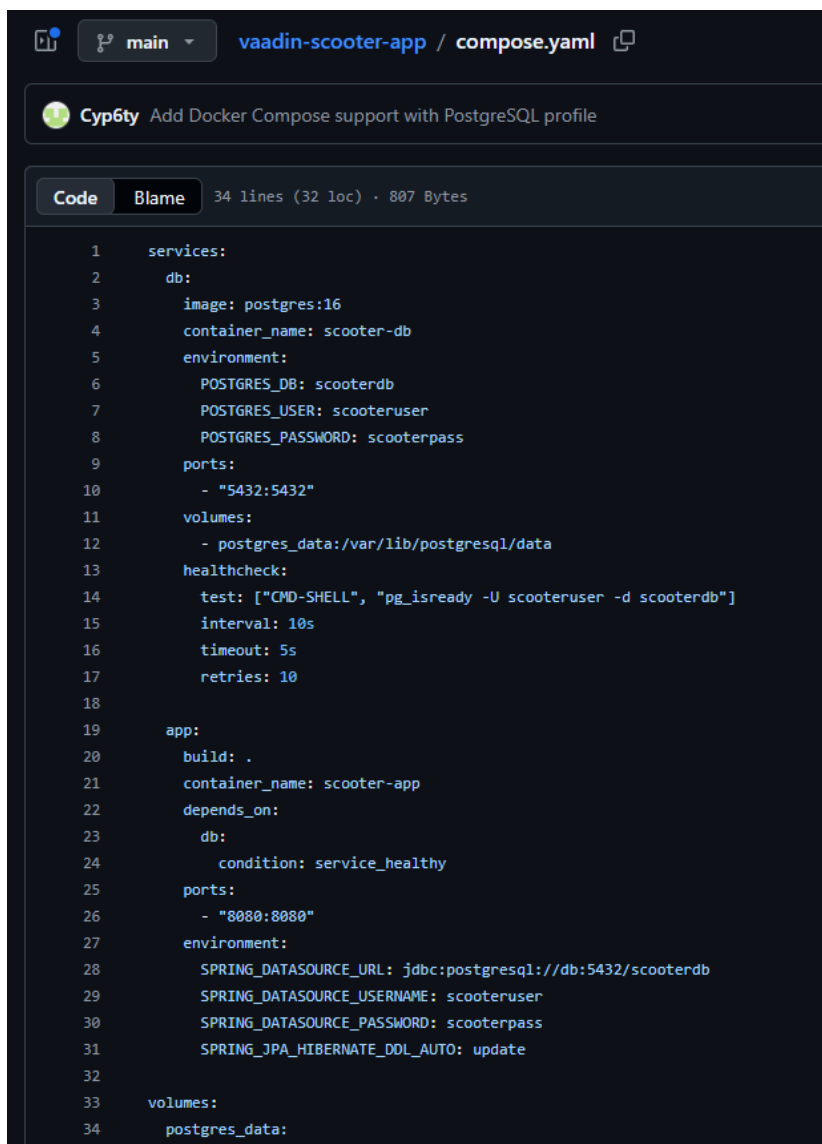
Toteutettiin myös compose.yaml-tiedosto, jonka avulla voidaan käynnistää koko sovelluskokoinaisuus yhdellä komennolla Docker Compose-ympäristössä. Compose-määrittelyyn luotiin kaksi erillistä palvelua: PostgreSQL-tietokanta sekä Vaadin / Spring Boot -sovellus. Sovelluspalvelu rakennetaan projektin Dockerfilen avulla, ja tietokantapalvelu käyttää valmiista PostgreSQL-imagea.

Docker Compose -tiedostossa määriteltiin sovellukselle tarvittavat tietokantayhteyden asetukset ympäristömuuttujilla, kuten tietokannan osoite, käyttäjätunnus ja salasana. Lisäksi tietokannalle määriteltiin oma pysyvä volume, jotta tietosisältö voidaan säilyttää konttien uudelleenkäynnistyksissä. Sovellus asetettiin käynnistymään vasta tietokantapalvelun jälkeen.

Toimivuus testattiin komennolla: `docker compose up --build`

Tämän jälkeen PostgreSQL-tietokanta ja Vaadin-sovellus käynnistyivät onnistuneesti Docker-konteissa, ja sovellus oli käytettävissä selaimessa portin 8080 kautta.

Docker Compose-ympäristössä käytettiin erillistä PostgreSQL-tietokantaa, joten paikallisessa H2-kehitysympäristössä ollut data ei siirtynyt automaattisesti mukaan. Sovellus käynnistyi tämän vuoksi puhtaalla tietokantasisällöllä, mikä oli odotettu lopputulos.

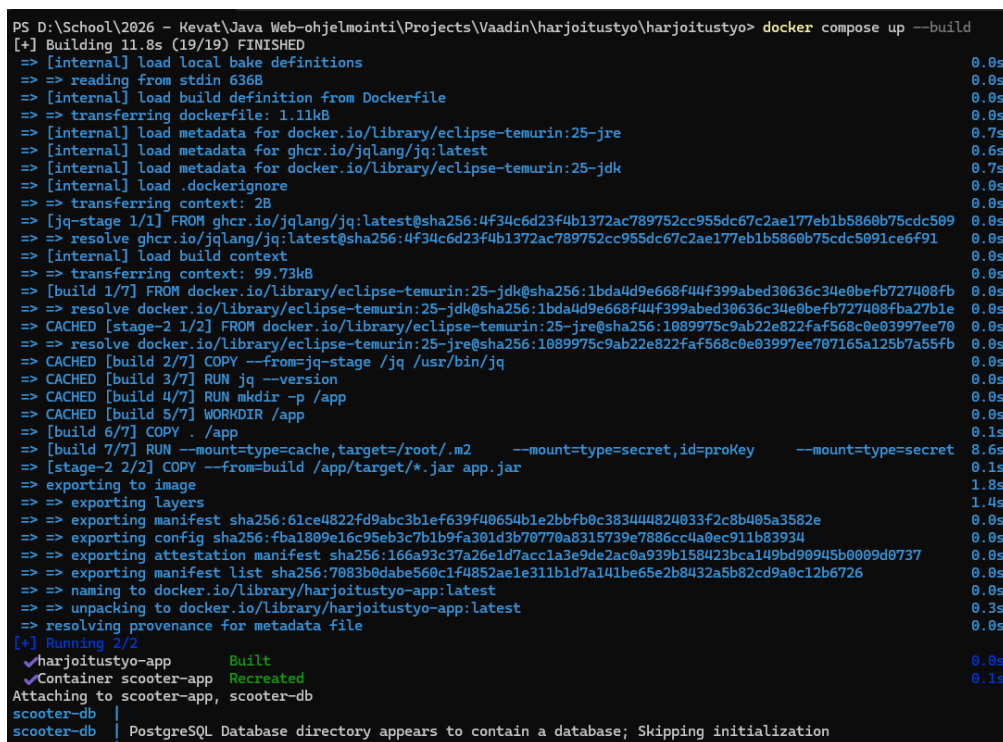


```

1  services:
2    db:
3      image: postgres:16
4      container_name: scooter-db
5      environment:
6        POSTGRES_DB: scooterdb
7        POSTGRES_USER: scooteruser
8        POSTGRES_PASSWORD: scooterpass
9      ports:
10       - "5432:5432"
11     volumes:
12       - postgres_data:/var/lib/postgresql/data
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U scooteruser -d scooterdb"]
15       interval: 10s
16       timeout: 5s
17       retries: 10
18
19     app:
20       build: .
21       container_name: scooter-app
22       depends_on:
23         db:
24           condition: service_healthy
25       ports:
26         - "8080:8080"
27       environment:
28         SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/scooterdb
29         SPRING_DATASOURCE_USERNAME: scooteruser
30         SPRING_DATASOURCE_PASSWORD: scooterpass
31         SPRING_JPA_HIBERNATE_DDL_AUTO: update
32
33     volumes:
34       postgres_data:

```

Kuva 71. compose.yaml GitHub-repositoriossa.

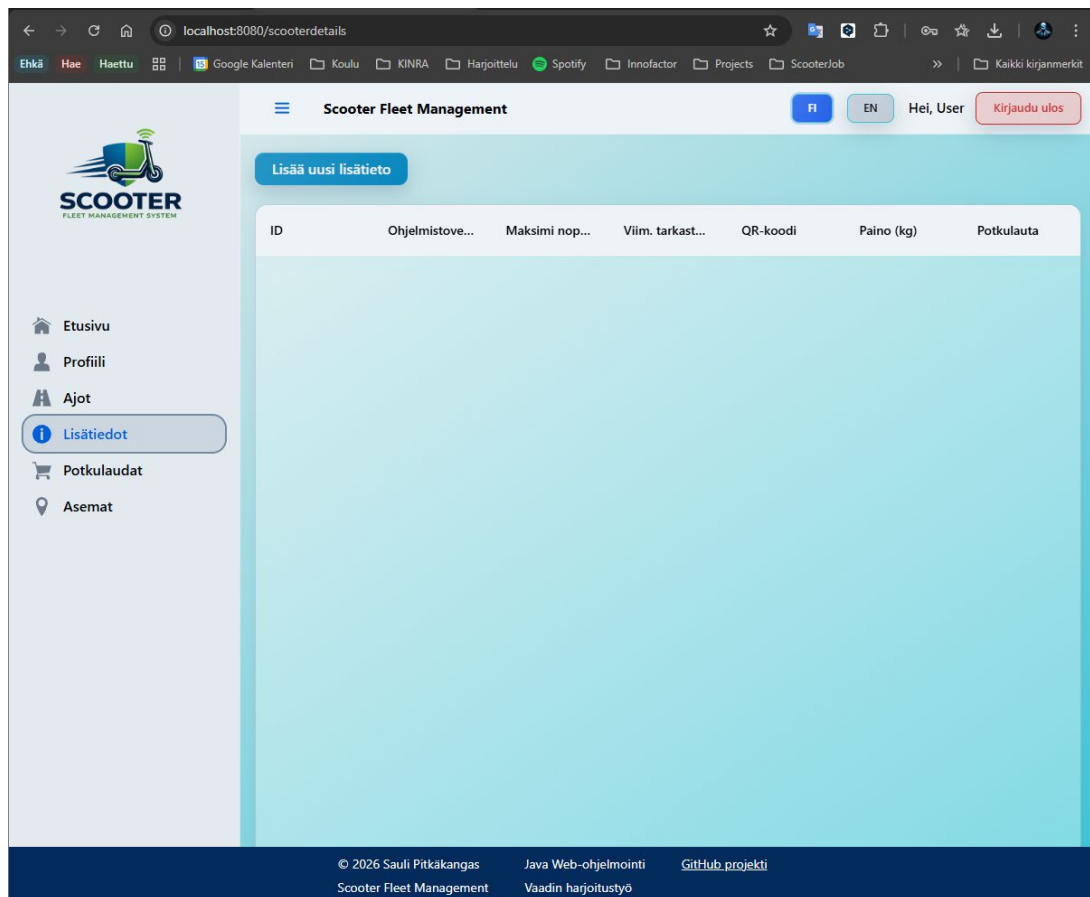


```

PS D:\School\2026 - Kevät\Java Web-ohjelmointi\Projects\Vaadin\harjoitustyo\harjoitustyo> docker compose up --build
[+] Building 11.8s (19/19) FINISHED
=> [internal] load local bake definitions                                0.0s
=> => reading from stdin 636B                                           0.0s
=> [internal] load build definition from Dockerfile                     0.0s
=> => transferring dockerfile: 1.1kB                                     0.0s
=> [internal] load metadata for docker.io/library/eclipse-temurin:25-jre 0.7s
=> [internal] load metadata for ghcr.io/jqlang/jq:latest                0.6s
=> [internal] load metadata for docker.io/library/eclipse-temurin:25-jdk 0.7s
=> [internal] load .dockerignore                                         0.0s
=> => transferring context: 2B                                           0.0s
=> [jq-stage 1/1] FROM ghcr.io/jqlang/jq:latest@sha256:4f34c6d23f4b1372ac789752cc955dc67c2ae177eb1b5860b75cdc509 0.0s
=> => resolve ghcr.io/jqlang/jq:latest@sha256:4f34c6d23f4b1372ac789752cc955dc67c2ae177eb1b5860b75cdc5091ce6f91 0.0s
=> [internal] load build context                                         0.0s
=> => transferring context: 99.73kB                                       0.0s
=> [build 1/7] FROM docker.io/library/eclipse-temurin:25-jdk@sha256:1bda4d9e668f44f399abed30636c34e0befb727408fba27b1e 0.0s
=> => resolve docker.io/library/eclipse-temurin:25-jdk@sha256:1bda4d9e668f44f399abed30636c34e0befb727408fba27b1e 0.0s
=> CACHED [stage-2 1/2] FROM docker.io/library/eclipse-temurin:25-jre@sha256:1089975c9ab22e822faf568c0e03997ee70 0.0s
=> => resolve docker.io/library/eclipse-temurin:25-jre@sha256:1089975c9ab22e822faf568c0e03997ee70 0.0s
=> CACHED [build 2/7] COPY --from=jq-stage /jq /usr/bin/jq              0.0s
=> CACHED [build 3/7] RUN jq --version                                  0.0s
=> CACHED [build 4/7] RUN mkdir -p /app                                 0.0s
=> CACHED [build 5/7] WORKDIR /app                                       0.0s
=> [build 6/7] COPY . /app                                              0.1s
=> [build 7/7] RUN --mount=type=cache,target=/root/.m2 --mount=type=secret,id=proKey --mount=type=secret 8.6s
=> [stage-2 2/2] COPY --from=build /app/target/*.jar app.jar           0.1s
=> exporting to image                                                  1.8s
=> => exporting layers                                                  1.4s
=> => exporting manifest sha256:61ce4822fd9abc3b1ef639f40654b1e2bbfb0c383444824033f2c8b405a3582e 0.0s
=> => exporting config sha256:fba1809e16c95eb3c7b1b9fa301d3b70770a8315739e7886cc4a0ec911b83934 0.0s
=> => exporting attestation manifest sha256:166a93c37a26e1d7acc1a3e9de2ac0a939b158423bca149bd90945b0009d0737 0.0s
=> => exporting manifest list sha256:7083b0dabe560c1f4852ae1e311bd7a141be65e2b8432a5b82cd9a0c12b6726 0.0s
=> => naming to docker.io/library/harjoitustyo-app:latest              0.0s
=> => unpacking to docker.io/library/harjoitustyo-app:latest          0.3s
=> => resolving provenance for metadata file                          0.0s
[+] Running 2/2
✔ harjoitustyo-app Built 0.0s
✔ Container scooter-app Recreated 0.1s
Attaching to scooter-app, scooter-db
scooter-db |
scooter-db | PostgreSQL Database directory appears to contain a database; Skipping initialization
scooter-db |

```

Kuva 72. docker compose up --build komento onnistuu terminaalisssa.



Kuva 73. Sovellus käynnissä selaimessa Docker Compose -ympäristössä.

9.5 Tiedoston lataus ja tallennus

Sovelluksen tiedoston lataus ja tallennus -toiminnallisuus toteutettiin käyttäjäprofiiliin profiilikuvan lisäyksen yhteydessä, josta tarkemmin kohdassa [Oman kuvan lisääminen](#). Käyttäjä voi ladata kuvatiedoston käyttöliittymän Upload-komponentilla, minkä jälkeen tiedosto tallennetaan palvelimen tiedostojärjestelmään pysyvästi.

Ladattu tiedosto liitetään käyttäjän tietoihin tallentamalla kuvan sijaintipolku tietokantaan. Käyttäjän lataama kuva voidaan hakea ja näyttää automaattisesti myöhemmillä kerroilla.

10 POHDINTA

Tämä harjoitustyö oli mahdollisesti laajin ja haastavin projektikokonaisuus, jonka olen toteuttanut yksin ilman luokkalaisten/tiimiläisten apua Full stack projektina. Työnaikana pääsi yhdistämään kursilla läpikäytyjä asioita, kuten tietokantojen käytön, JPA-entiteetit, relaatiot, CRUD-toiminnot, käyttöliittymien rakentamisen sekä sovelluksen rakenteellisen suunnittelun. Tämä projekti auttoi ymmärtämään ja käsittämään selvemmin, miten yksittäiset opitut osa-alueet liittyvät toisiinsa käytännön sovelluskehityksessä.

Työn suurin haaste oli kokonaisuuden hahmottaminen sovelluksen suunnittelu vaiheessa. Kun sovelluksessa on useita entiteettejä, näkymiä ja toiminnallisuuksia, pelkkä koodin kirjoittaminen ei riitä, vaan myös rakenteen suunnittelu korostui. Kurssilla käytiin useasti eri esimerkkejä asiakkaiden välisistä suhteista ja niihin liittyvistä asioista, ja tämän projektin toteutukseen ei annettu valmiiksi valmista aihetta. Itse sovelluksen suunnittelu ja miettiminen vei useamman päivän, kun en halunnut kuitenkaan tehdä asiakassovellusta, vaan kehittää jotain omaperäistä. Halusin yrittää soveltaa opittuja asioita asiakassuhteista parhaani mukaan omaan aiheeseeni, joka tuotti paljon miettimistä ja epäkäytännöllisiä tapoja, joihin kuitenkin loppupeleissä löytyi mielestäni sopivat ratkaisut. Projektin edetessä kuitenkin opin hahmottamaan, kuinka tärkeää on pitää koodi selkeänä ja jakaa vastuut esimerkiksi view-, service- ja repository-luokkien välillä. Tämä helpotti työn jatkamista myöhemmissä vaiheissa.

Olen tottunut rakentamaan käyttöliittymiä enemmän Reactilla ja TypeScriptilla, ja huomasin kuitenkin hyödylliseksi rakentaa myös käyttöliittymän Vaadinilla, vaikka se vähän erilaista olikin. Oli motivoivaa nähdä nopeasti konkreettisia tuloksia, kun uusia näkymiä, lomakkeita ja taulukoita sai näkyviin toimivana kokonaisuutena. Samalla opin käyttöliittymän käytettävyydestä, asetelusta ja visuaalisuudesta. Myös CSS-tyyliin muokkaaminen ja oman ilmeen rakentaminen toi projektiin lisää mielenkiintoa, kun pääsi hyödyntämään asioita, joita olen oppinut jo ennestään ennen tätä kurssia.

Teknisesti haastavimpia osuuksia olivat dynaaminen suodatus Criteria API:lla, tietoturvaosion rakentamisen kokonaisuudessaan sekä Docker ympäristön käyttöönotto. Näiden toteuttaminen vaati enemmän selvitystyötä ja aikaa, kuin perinteiset CRUD-toiminnot, mutta siinä samalla tuli tutuksi uusia ja hyödyllisiä taitoja sekä tapoja toteuttaa kyseiset toiminnallisuudet. Lisäksi projektin aikana tuli vastaan useita pieniä ja isompia virheitä sekä ongelmatilanteita, joiden ratkaiseminen vei oman aikansa. Tekoälyä käytin useasti opettamaan sekä selvittämään, miten tietyt tilanteet tulisi tehdä järkevästi, selvästi ja hyödyllisesti. Tämä auttoi myös kehittämään ongelmanratkaisukykyä ja kärsivällisyyttä.

Aivan kaikkia tehtävänannon lisäominaisuuksia en ehtinyt tai osannut toteuttaa. Esimerkiksi osa jatkokehitykseen liittyvistä ominaisuuksista jäi ajan puutteen vuoksi pois. Olen kuitenkin erittäin tyytyväinen lopputulokseen, vaikka sovellus ei vielä täysin valmis olekaan. Sain valmiiksi laajan ja toimivan kokonaisuuden, joka täyttää suuren osan tehtävänannon vaatimuksista ja sisältää useita ylimääräisiäkin ominaisuuksia. Projekti opetti myös sen, että kaikkea ei tarvitse tehdä heti täydellisesti, vaan että on tärkeää saada kokonaisuus valmiiksi, ja korjata vasta sitten pääasiallisten toiminnallisuuksien jälkeen rikki jääneet asiat kuntoon. Projektin edetessä itse sovellus, ajatukset, toteutus sekä toteutustavat muuttuivat, joten tietyt asiat voidaan tehdä kuntoon vasta sitten, kun hahmottaa kokonaisuuden.

Kokonaisuutena harjoitustyö oli erittäin positiivinen ja opettavainen kokemus. Työn rakentamisen jälkeen tajusin paremmin Java-pohjaisen web-sovelluksen rakennetta, tietokannan ja käyttöliittymän yhteistyötä sekä projektin vaiheittaista kehittämistä. Uskon myös, että tästä projektista saatu osaaminen auttaa tulevaisuudessa tulevissa opinnoissa sekä myöhemmin työelämässä.

Tavoitteeni, johon tähtäsin, oli saada kurssista ja tästä harjoitustyöstä numero 5. Valitettavasti huomasin liian myöhään, että minulta oli jäänyt tekemättä Tietoturva-osiossa olevan Gmail ja GitHub-tunnuksilla kirjautuminen. Laskin kuitenkin, että 30 vaadittua kohtaa olisi täyttynyt.

Alla vielä lista poisjääneistä toiminnallisuuksista sovelluksessa tehtävänantoon viitaten:

1. Lisää mahdollisuus autentikoinnille, niin että siihen voi käyttää Gmail tai GitHub tunnusta.
2. Toteuta Vaadin Server Push.
3. Sähköpostin lähetys ylläpitokäyttäjälle, kun luodaan uusi käyttäjä.
4. Salasanan vaihto sähköpostin avulla.
5. CSV- tai Excel-datan tuonti ja vienti.
6. Tutustu Spring Auditingiin ja lisää sovellukseen auditointi.
7. Historiatiedot eli jokaisesta entiteetin muutoksesta jää historiatieto.
8. Historiatiedon näyttäminen aikajanalla käyttöliittymällä tai lista entiteetin historiasta.
9. Lisää ulkoinen javascript pohjainen komponentti sovellukseen, kuten Quill.js.

LÄHTEET

OpenAI. 2026. ChatGPT. GPT-5.3. Käytetty ohjelmointiongelmien ratkaisemiseen, rakenteen suunnitteluun, tekstin muotoiluun ja raportin kielentarkistukseen ja korjaamiseen harjoitustyön aikana.
<https://chat.openai.com>

Vaadin Ltd 2026. Vaadin Documentation. Saatavilla: <https://vaadin.com/docs>

VMware 2026. Spring Documentation. Saatavilla: <https://spring.io/projects>

Docker Inc. 2026. Docker Documentation. Saatavilla: <https://docs.docker.com>